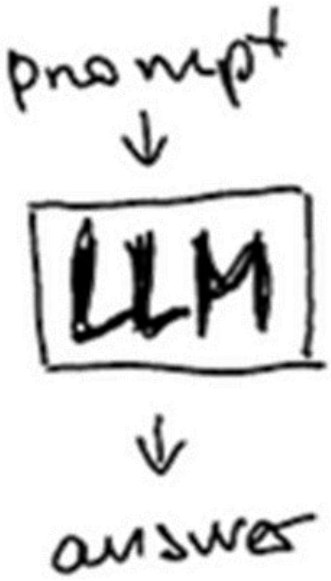
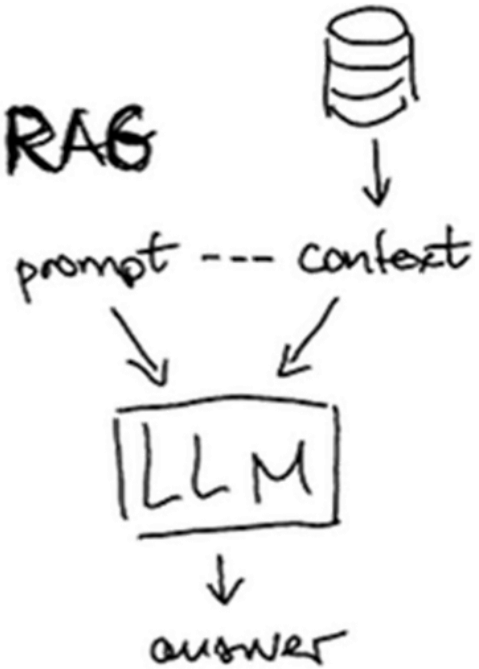
LangChain agents and tools

Recall chains

```
chain = (
    {"text": RunnablePassthrough()}
    | prompt
    | llm
    | output_parser
)
```

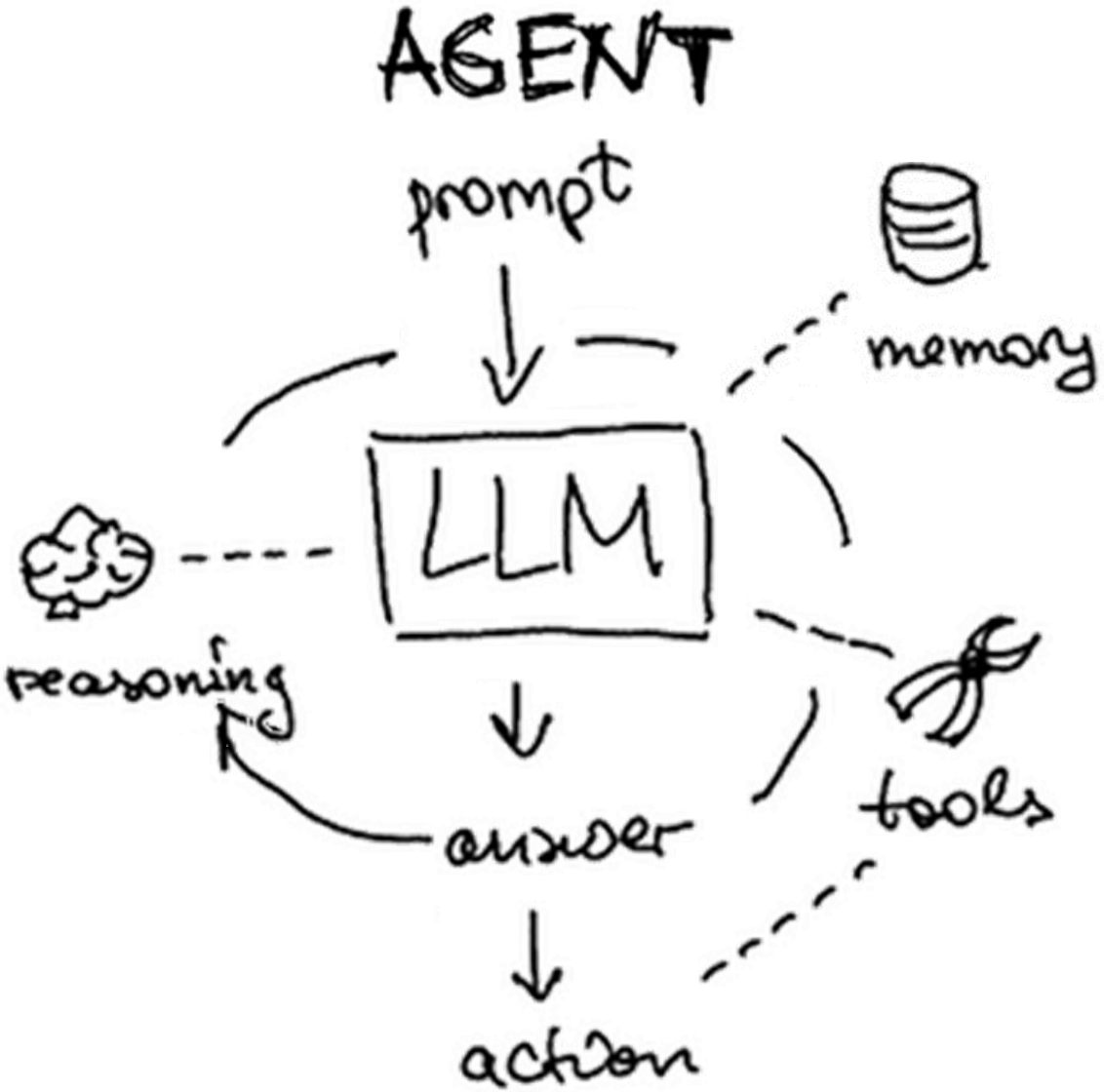



```
rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
```



- But, what if a simple chain is insufficient for your application?
  - Applications requiring iteration and reasoning based on results

LangChain Agents and Tools



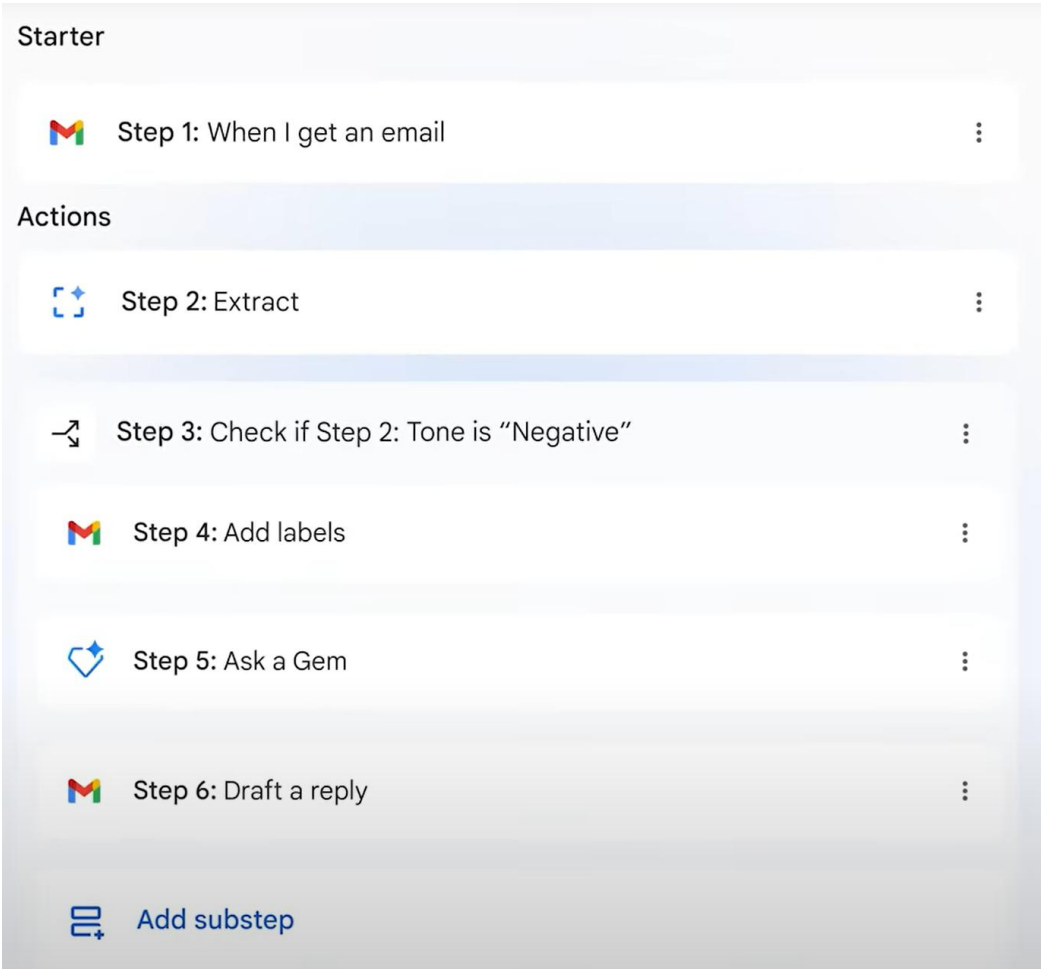
Agents

- Definition
  - A program in which the output of an LLM impacts its execution flow
  - LLM is brain, program is vehicle it drives

Examples

Google Workspace Studio is here to let anyone build custom AI agents with zero coding required

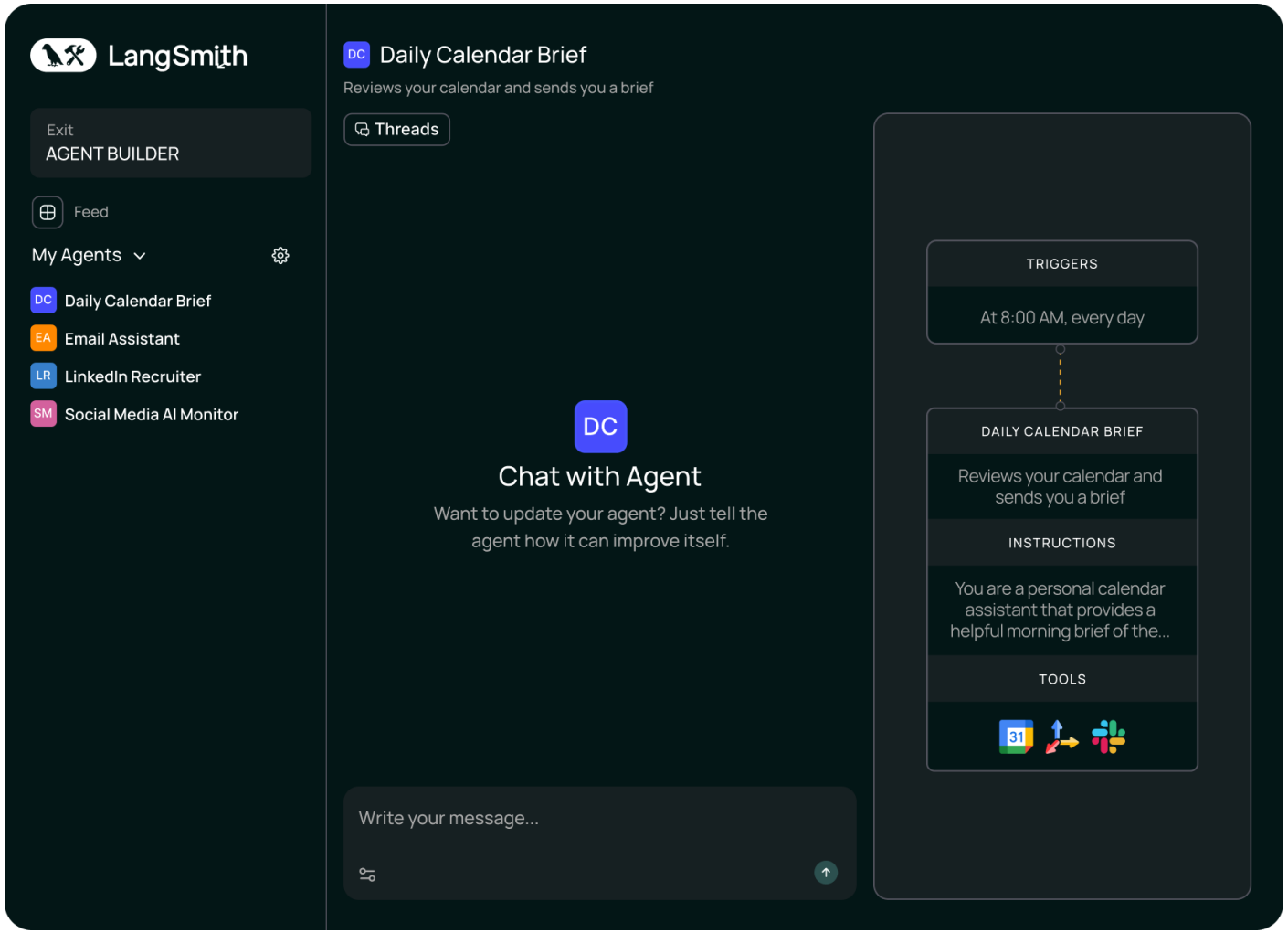
December 4, 2025 By Robby Payne | View Comments



LangChain

LangSmith Agent Builder

No-code agents for real work



Anthropic

REECE ROGERS GEAR JAN 15, 2026 12:40 PM

Anthropic's Claude Cowork Is an AI Agent That Actually Works

Cowork is a user-friendly version of Anthropic's Claude Code AI-powered tool that's built for file management and basic computing tasks. Here's what it's like to use it.

- But…

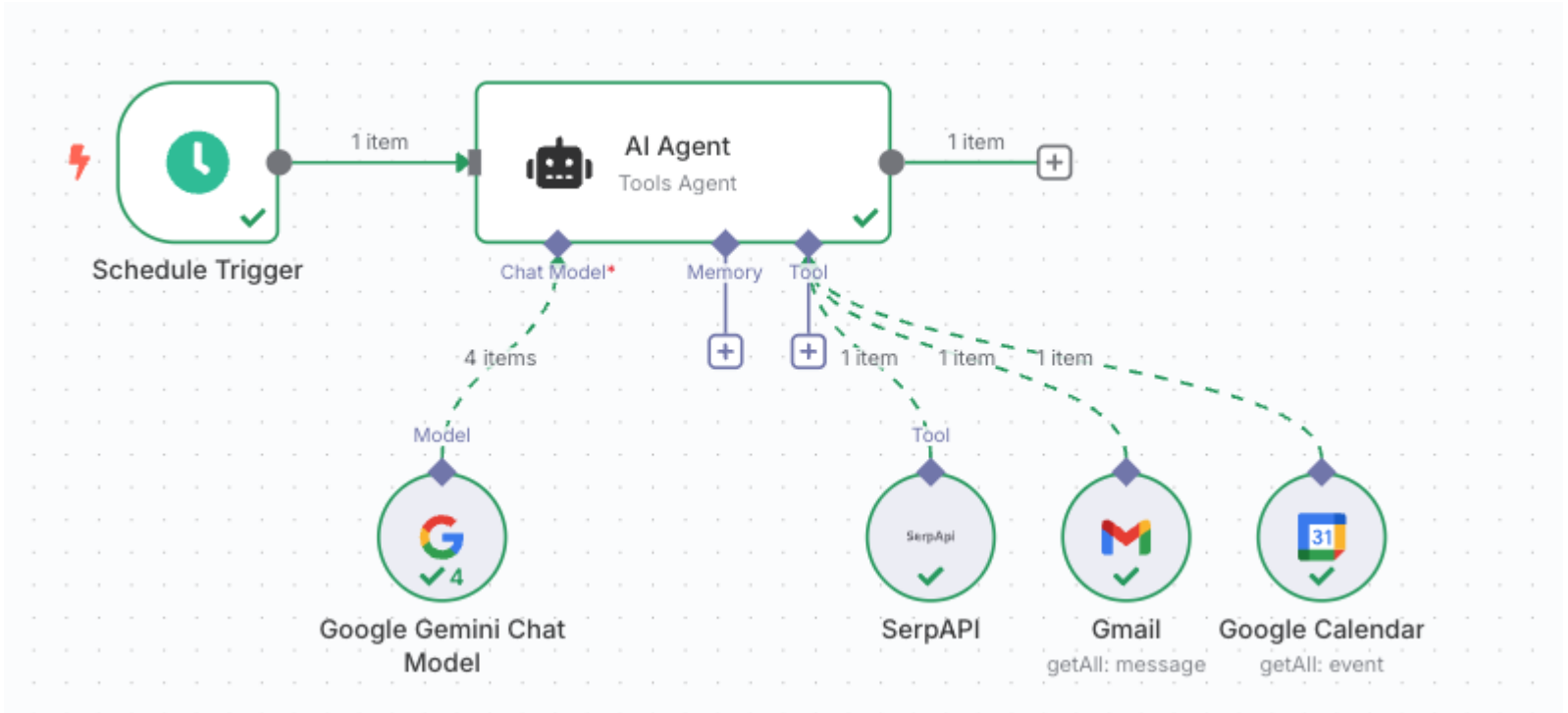
  The biggest reason for not trying out Cowork is the ongoing security risk inherent in these kinds of agents. Like most agents, Cowork is susceptible to prompt injection attacks, secret messages hidden online that try to trick AI tools and deviate them from tasks. You shouldn't expose sensitive data to a tool that can be compromised in this way.

n8n

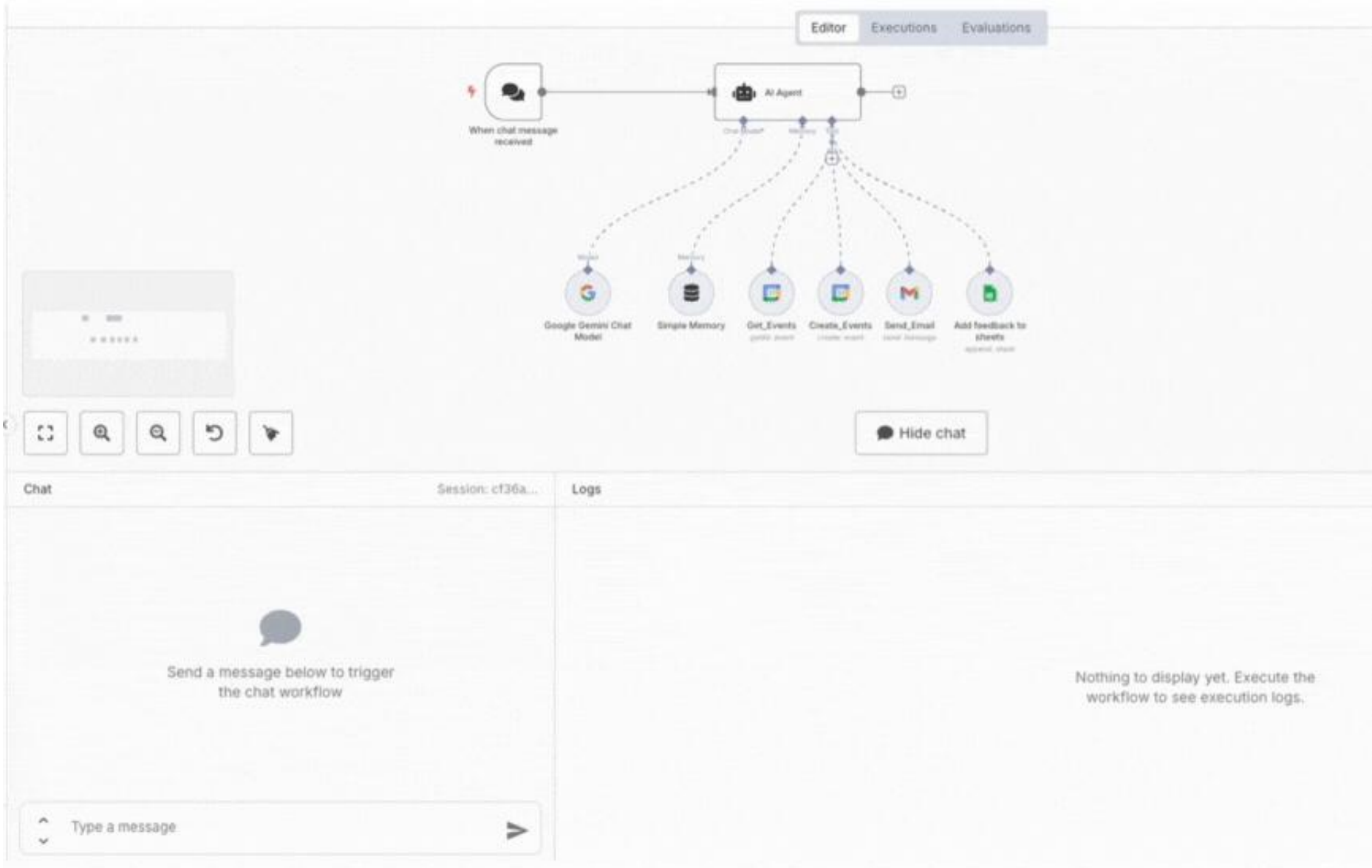
Building AI-Powered Low-Code Workflows with n8n

ALESSANDRA COSTA

Jun 23, 2025 • 27 min read



- Client intake chatbot



Agency levels

"Levels" of agency

- None
  - LLM output has no impact on program flow
- Router (Shuster 2022)
  - LLM output determines basic control flow

```
if llm_decision(): path_a() else: path_b()
```

- Tool calling (Nakano 2022)
  - LLM output determines function execution

```
tool, args = llm_function_selection_args_generation(goal)
run_function(tool, args)
```

- Multi-step agent (Yao 2023)
  - LLM output controls iteration and program continuation

```
while llm_continuation_decision():
  generate_and_execute_next_step()
```

- Multi-agent (Guo 2024)
  - LLM output invoking multiple other agents to complete task

```
if llm_decision():
    execute_agent()
```

- Code agent (Wang 2024)
  - LLM writes and executes code, including new tools if needed

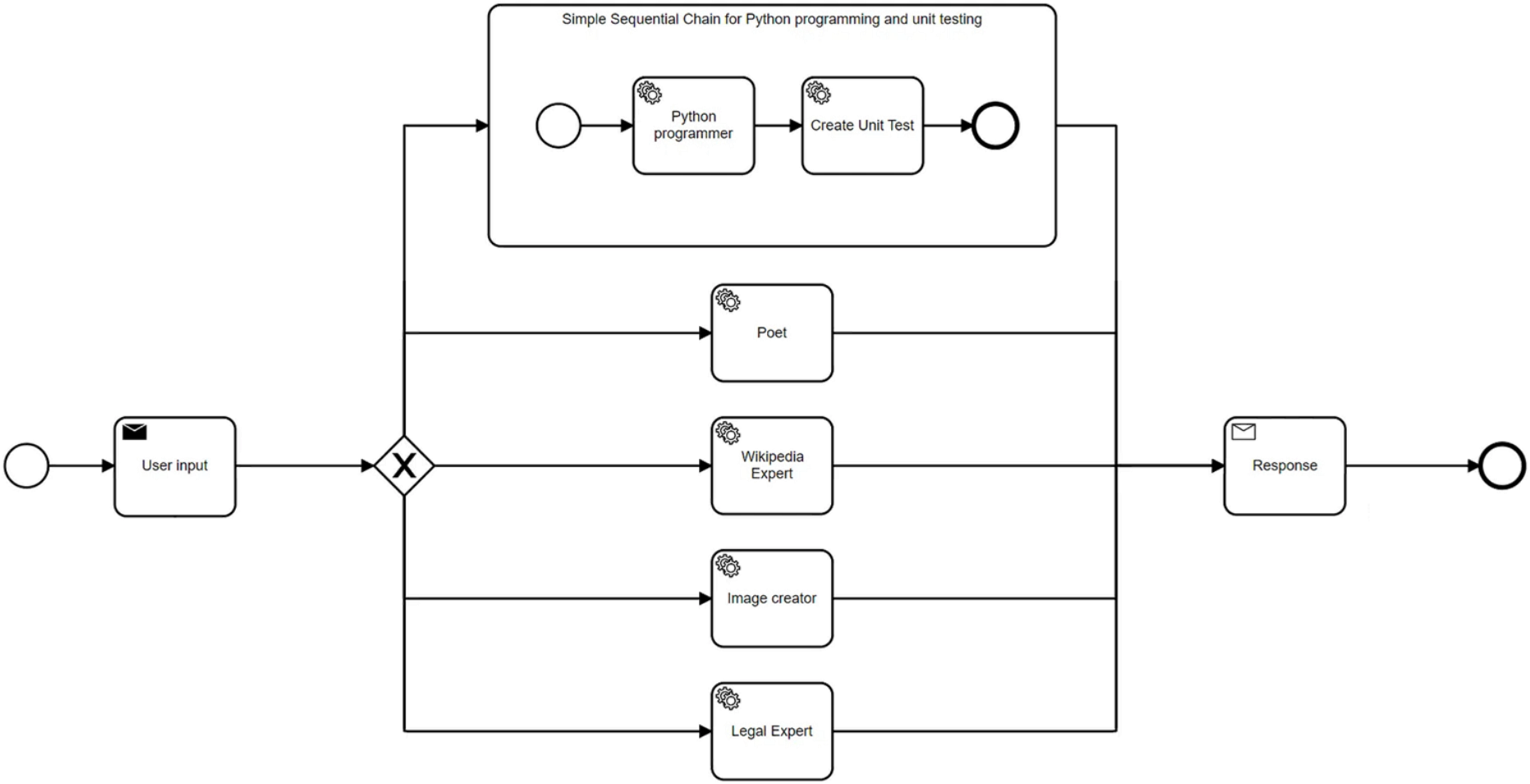
No agency

- LLM as a function

```
response = llm.invoke("Write me a haiku about Bulbasaur")
print(response.content)
```

Router

- Use LLM to help forward request to appropriate model, prompt template, or code execution path



Tool-calling

- Allow the LLM to select and invoke pre-defined tools
- Two code execution tools: Python, Javascript

```
tools = [command.ExecPython(), command.ExecJavaScript()]
agent = create_tool_calling_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

agent_executor.invoke({"input": "print('hello')"})
```

> *Entering new AgentExecutor chain…*
> *Invoking: `riza_exec_python` with `{'code': "print('hello')"}`*
> *hello*
> *The Python code has been executed and produced the output "hello".*

```
agent_executor.invoke({"input": "console.log('hello')"})
```

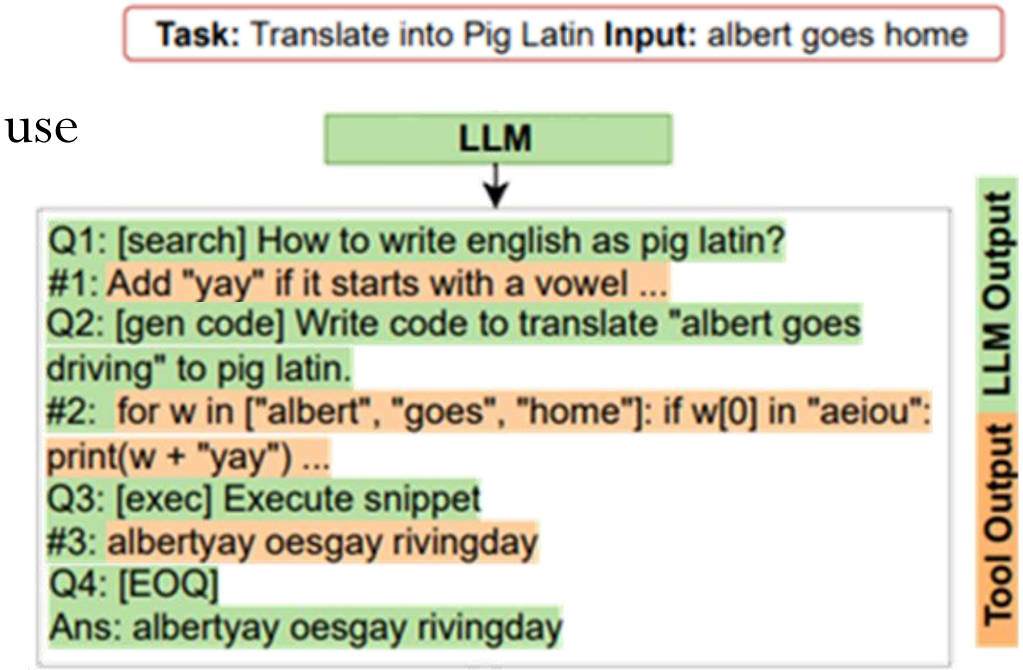
> *Entering new AgentExecutor chain…*
> *Invoking: `riza_exec_javascript` with `{'code': "console.log('hello');"}`*
> *hello*
> *The provided JavaScript code simply prints out "hello" to the console.*

- Most models now support native tool-calling (`.bind_tools`)

Multi-step agents

Approach

- Allow LLM to generate and orchestrate tool calls
- Automatic Reasoning and Tool Use ART (Paranjape 2023)
  - Library of tools available for use
    - `[search]`
    - `[gen_code]`
    - `[exec]`
  - LLM generates multi-step reasoning
  - Pauses generation to call external tools
  - Integrates tool responses and resumes generation



Example

- Prompt template

```
Here is a question: "{question}"
You have access to these tools: {tools_descriptions}.
You should first reflect with 'Thought: {your_thoughts}', then you either:
- call a tool with the proper JSON formatting,
- or you print your final answer starting with the prefix 'Final Answer:'
```

- Input question

```
How many seconds are in 1:23:45?
```

- Prompt

```
Here is a question: "How many seconds are in 1:23:45?"
You have access to these tools:
    - convert_time: converts a time given in hours:minutes:seconds into seconds.

You should first reflect with 'Thought: {your_thoughts}', then you either:
- call a tool with the proper JSON formatting,
- or you print your final answer starting with the prefix 'Final Answer:'
```

```
Here is a question: "How many seconds are in 1:23:45?"
You have access to these tools:
    - convert_time: converts a time given in hours:minutes:seconds into seconds.

You should first reflect with 'Thought: {your_thoughts}', then you either:
- call a tool with the proper JSON formatting,
- or you print your final answer starting with the prefix 'Final Answer:'
```

```
Thought: I need to convert the time string into seconds.
```

```
Action:
{
    "action": "convert_time",
    "action_input": {
        "time": "1:23:45"
    }
}
```

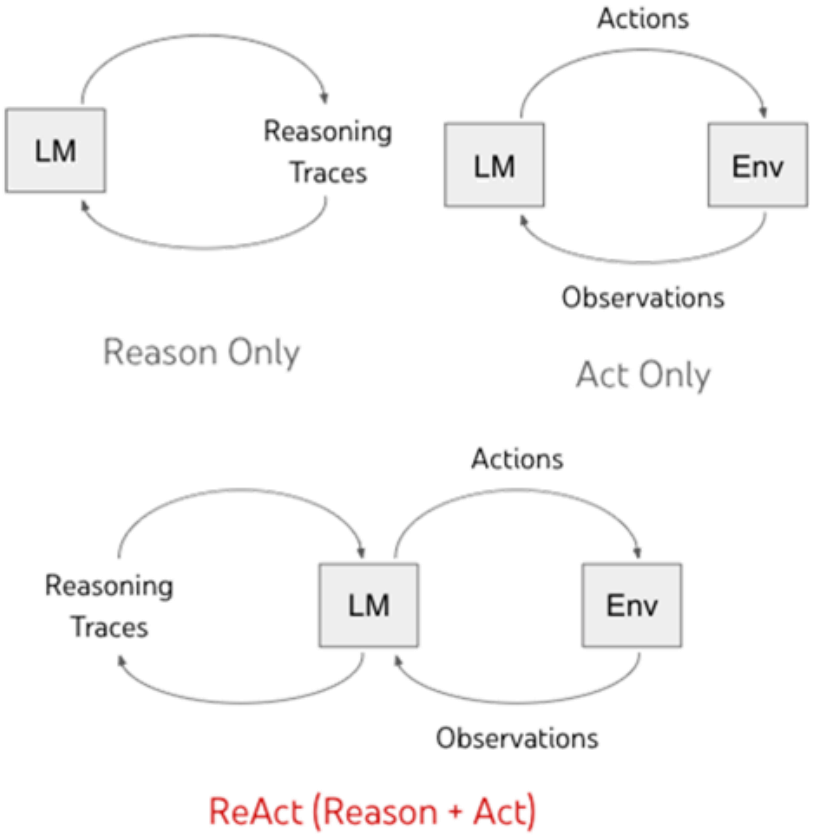
```
Observation: {'seconds': '5025'}
```

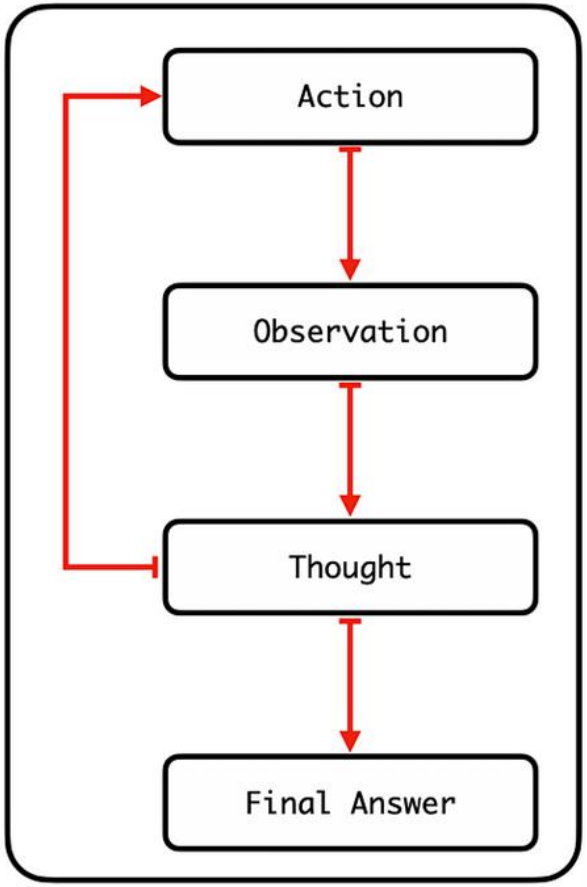
```
Thought: I now have the information needed to answer the question.
```

```
Final Answer: There are 5025 seconds in 1:23:45.
```

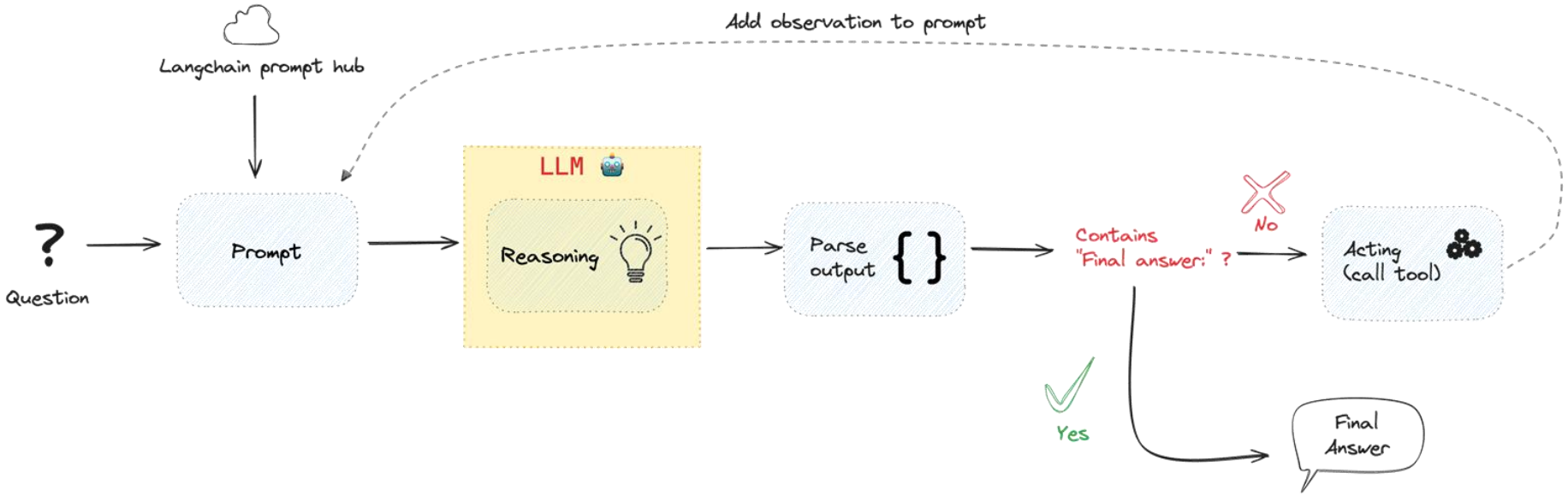
ReAct: Re(ason)/Act(ion) agent

- ReAct (Yao et al. 2022)
- Chain-of-thought reasoning with actions and observations
  - Performs reasoning, action, and observation steps in a loop until final answer calculated
  - Similar to how humans do it!





- Utilized in conjunction with access to external world
  - Basis of one of this week's agent exercises



ReAct prompt template example

```
Answer the following questions as best you can. You have access to
the following tools:

Wikipedia: A wrapper around Wikipedia.  Useful when you need to
answer general questions about people, places, companies, facts,
historical events or other subjects. Input should be a search
query.
Calculator: Useful for when you need to answer questions about
math.

The way you use the tools is by specifying a json blob.
Specifically, this json should have an `action` key (with the name
of the tool to use) and a `action_input` key (with the input to the
tool going here).

The only values that should be in the "action" field are:
Wikipedia, Calculator

The $JSON_BLOB should only contain a SINGLE action. Here is an
example of a valid $JSON_BLOB:
{
  "action": $TOOL_NAME,
  "action_input": $INPUT
}
```

```
ALWAYS use the following format:

Question: the input question you must answer
Thought: you should always think about what to do
Action:
$JSON_BLOB
Observation: the result of the action
... (this Thought/Action/Observation can repeat N times)
Thought: I now know the final answer
Final Answer: the final answer to the original input question
Begin!
```

Example execution

Answer the following questions as best you can. You have access to the following tools:

Wikipedia: A wrapper around Wikipedia. Useful for when you need to answer general questions about people, places, companies, facts, historical events, or other subjects. Input should be a search query.
Calculator: Useful for when you need to answer questions about math.

Tools

Use the following format:

Question: the input question you must answer
Thought: you should always think about what to do
Action: the action to take, should be one of [Wikipedia, Calculator]
Action Input: the input to the action
Observation: the result of the action
... (this Thought/Action/Action Input/Observation can repeat N times)
Thought: I now know the final answer
Final Answer: the final answer to the original input question

ReAct format

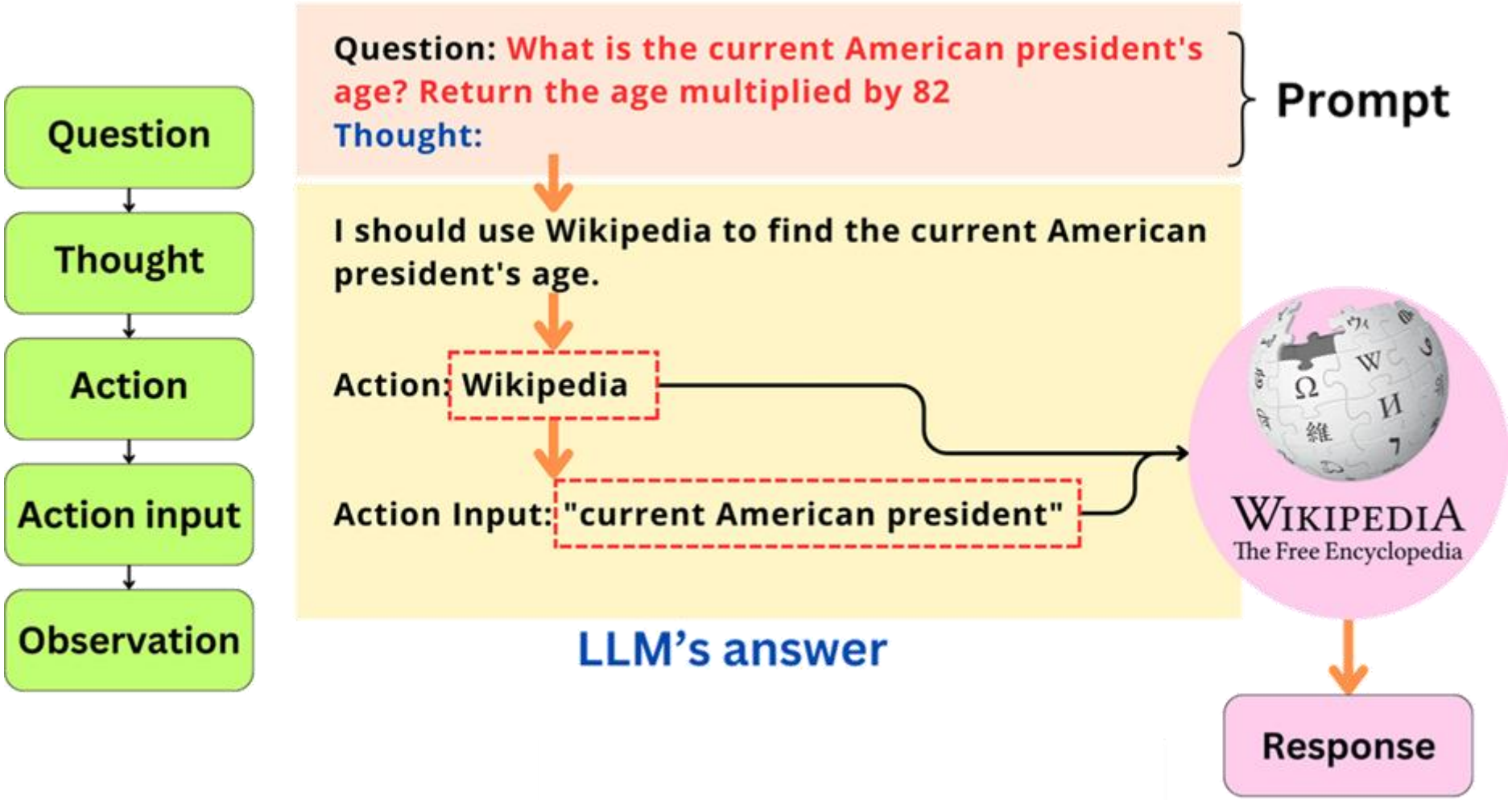
Begin!

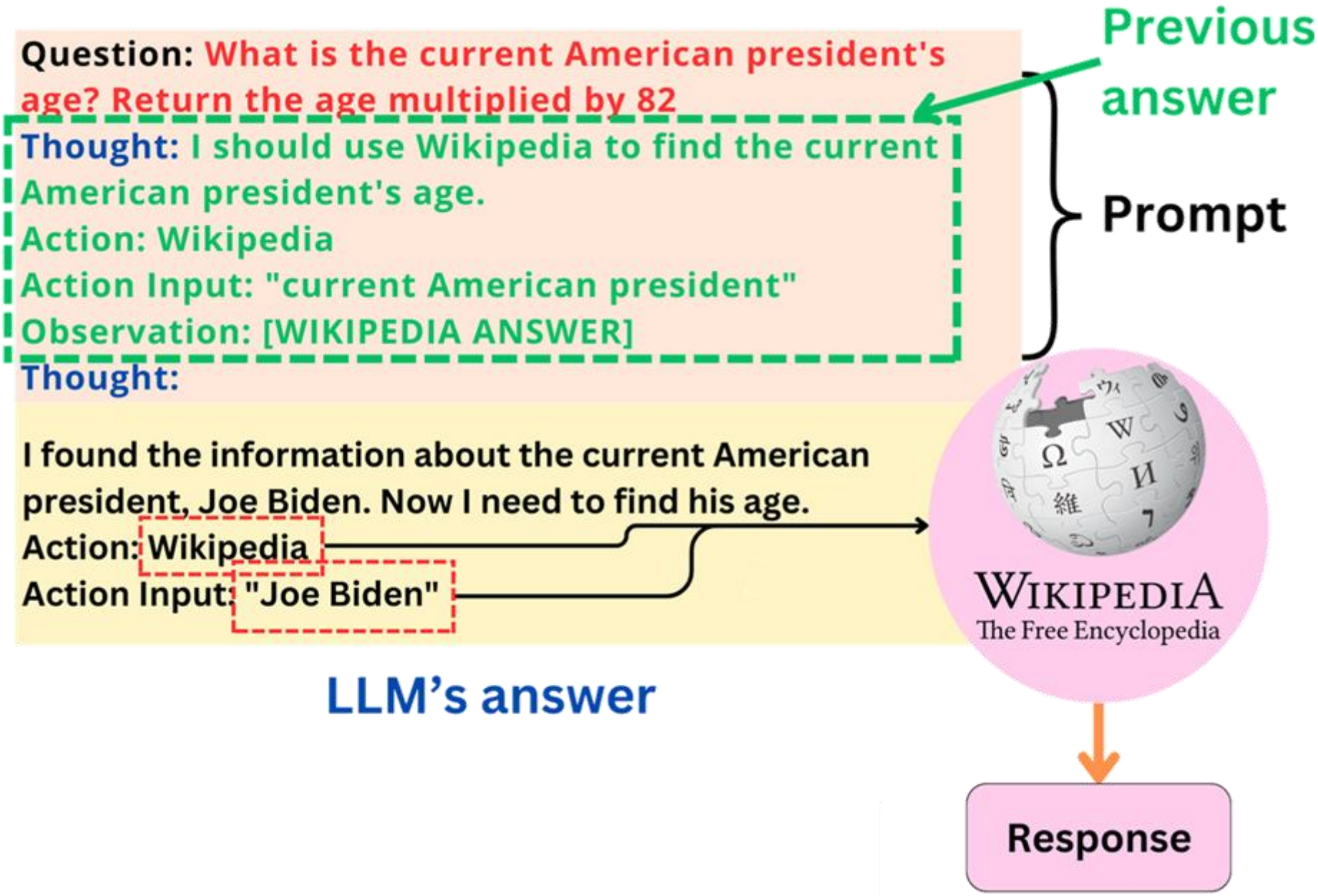
Question

Question: What is the current American president's age? Return the age multiplied by 82

Thought:

Output indicator

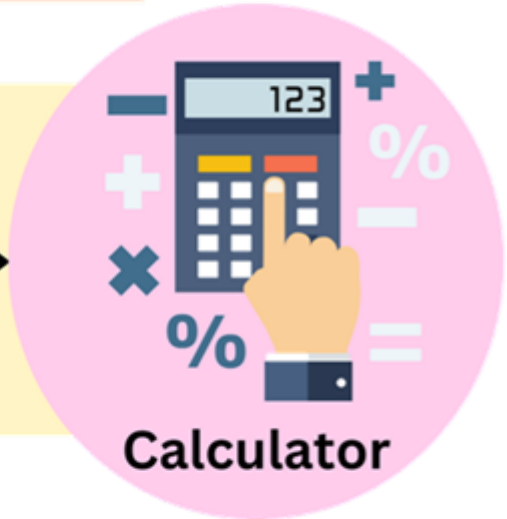

Question
Thought
Action
Action input
Observation
Question: What is the current American president's age? Return the age multiplied by 82
Thought:
Prompt
I should use Wikipedia to find the current American president's age.
Action: Wikipedia
Action Input: "current American president"
WIKIPEDIA
The Free Encyclopedia
LLM's answer
Response


Question: What is the current American president's age? Return the age multiplied by 82
Thought: I should use Wikipedia to find the current American president's age.
Action: Wikipedia
Action Input: "current American president"
Observation: [WIKIPEDIA ANSWER]
Thought:
Previous answer
Prompt
I found the information about the current American president, Joe Biden. Now I need to find his age.
Action: Wikipedia
Action Input: "Joe Biden"
WIKIPEDIA
The Free Encyclopedia
LLM's answer
Response

Question: What is the current American president's age? Return the age multiplied by 82
Thought: [FIRST ITERATION ANSWER]
Thought: [SECOND ITERATION ANSWER]
Thought:

Prompt

I found Joe Biden's age on Wikipedia. Now I can calculate his age multiplied by 82.
Action: Calculator
Action Input: 79 * 82



LLM's answer

Response

Question: What is the current American president's age? Return the age multiplied by 82
Thought: [FIRST ITERATION ANSWER]
Thought: [SECOND ITERATION ANSWER]
Thought: [THIRD ITERATION ANSWER]
Thought:

} Prompt

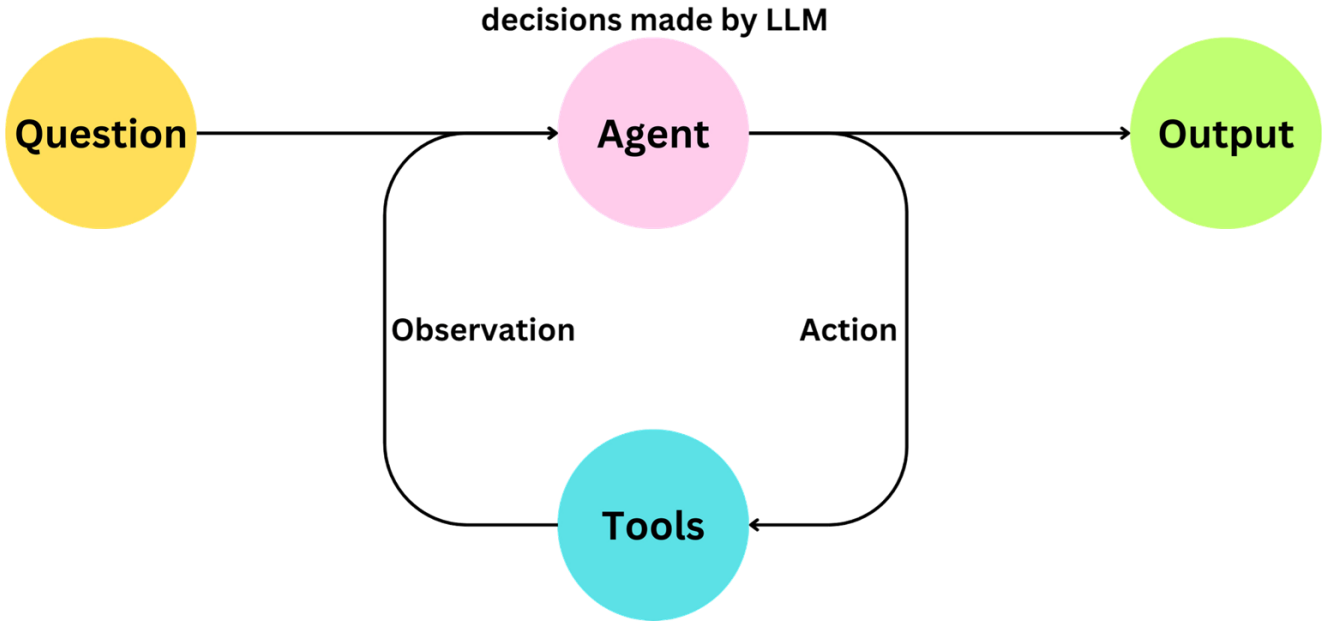
Ending condition

I now know the final answer
Final Answer: The current American president's age multiplied by 82 is 6478.

LLM's answer

Agents (`langchain.agents`)

- Library of multi-step agents combining tools, language models, and reasoning in a loop to perform complex operations



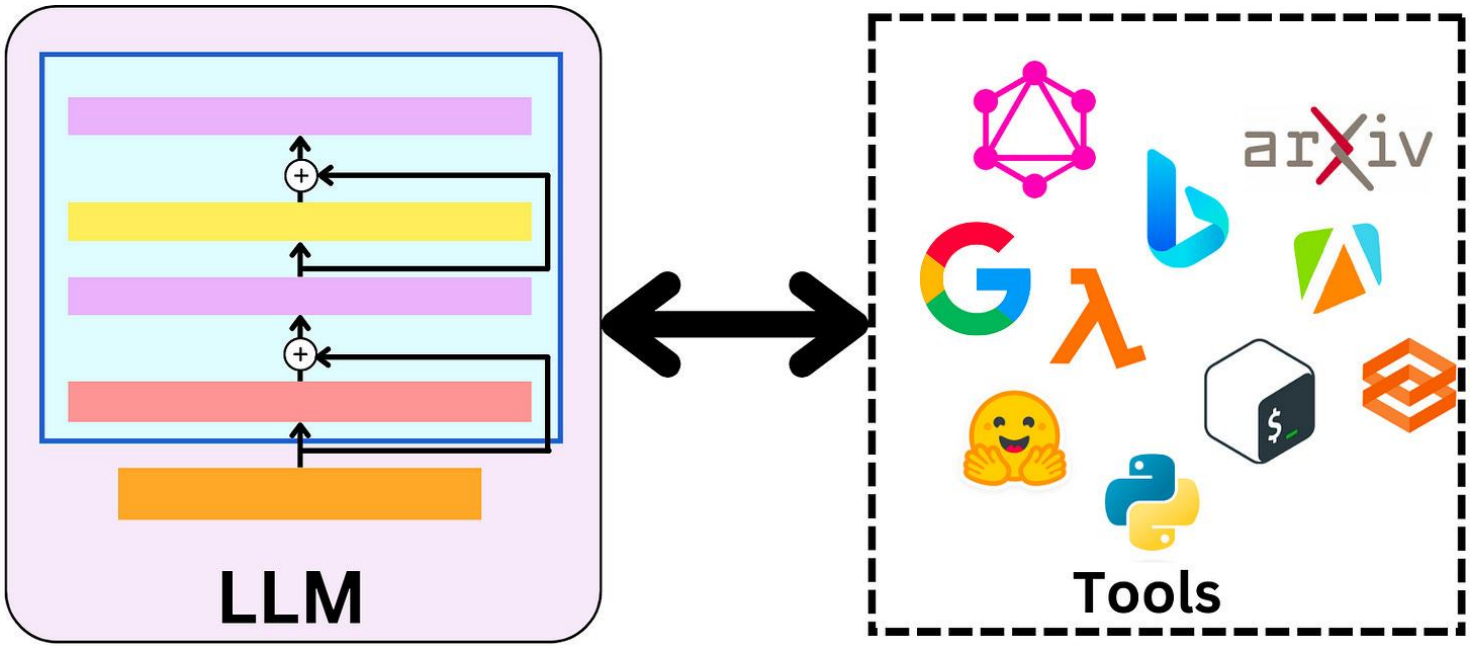
- Retrieve current data via `serpapi`, Perform math via `llm-math`

```
from langchain.agents import AgentExecutor
tools = load_tools(["serpapi", "llm-math"], llm=llm)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
agent_executor.invoke({
    "input": "Who is Leo DiCaprio's girlfriend? What is her current age raised
to the 0.43 power?"
})
```

```
> Entering new AgentExecutor chain...
     I need to find out who Leo DiCaprio's girlfriend is and then calculate
  her age raised to the 0.43 power.
    Action: Search
    Action Input: "Leo DiCaprio girlfriend"
    Observation: model Vittoria Ceretti
    Thought: I need to find out Vittoria Ceretti's age
    Action: Search
    Action Input: "Vittoria Ceretti age"
    Observation: 25 years
    Thought: I need to calculate 25 raised to the 0.43 power
    Action: Calculator
    Action Input: 25^0.43
    Observation: Answer: 3.991298452658078
    Thought: I now know the final answer
    Final Answer: Leo DiCaprio's girlfriend is Vittoria Ceretti and her
  current age raised to the 0.43 power is 3.991298452658078.
    > Finished chain.
```

Tools (`langchain.tools`)

- LLM application initialized with a set of calls (e.g. tools) that implement specific functions
  - Tools interact with the world to deliver information to application
- LLM, if it decides it can make progress answering your question by using a tool, will invoke it



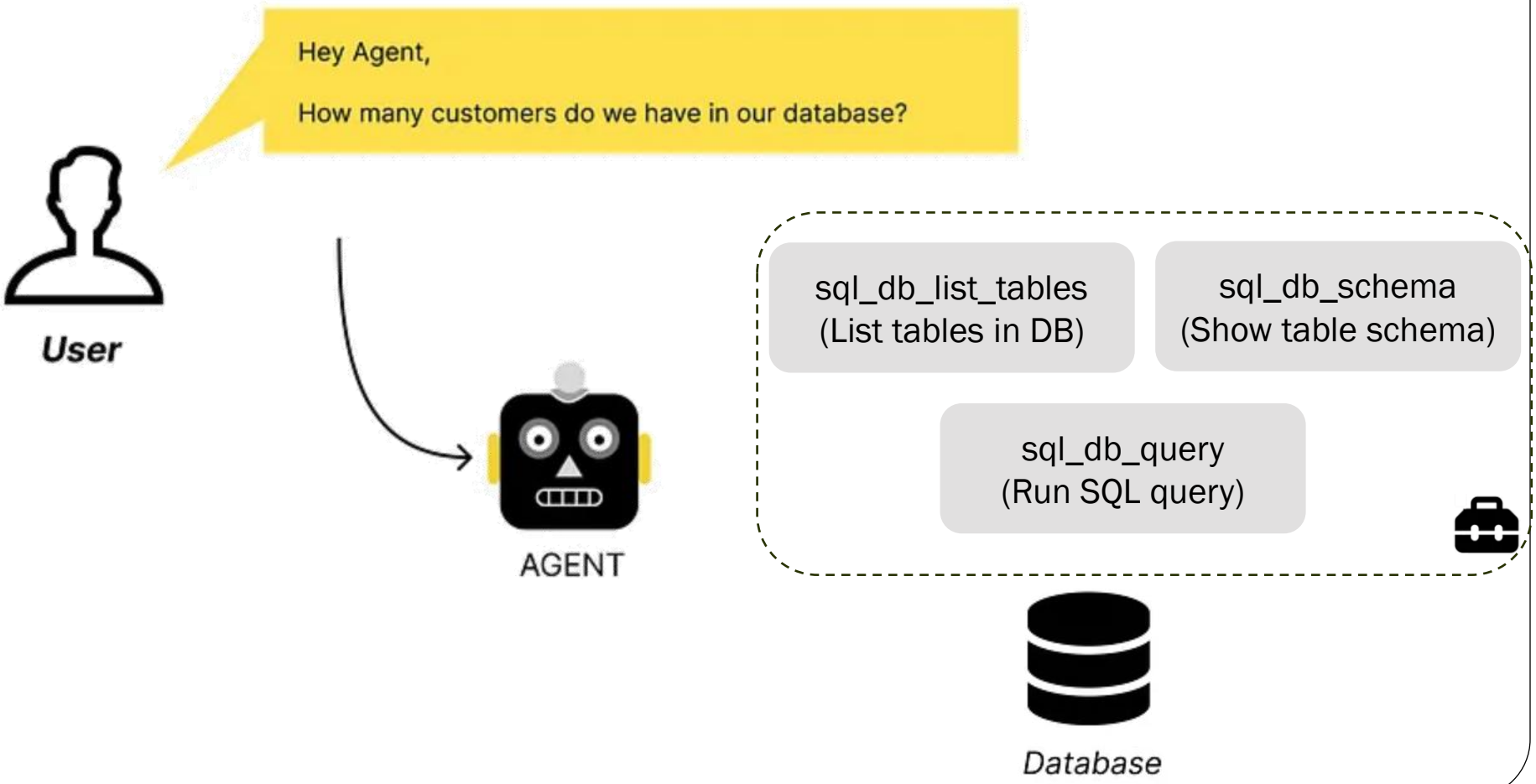
Built-in tools

- Terminal, File System
- Google Search, SerpAPI, DuckDuckGo Search, Wikipedia
- Apify, GraphQL
- Dall-E image generation
- Text to speech, Google Lens
- Google Drive
- OpenWeatherMap
- PubMed
- SQL database
- Twilio

- Tools must return information compactly since result goes back to LLM

Toolkits

- Collections of related Tools often used together
- Example: SQLDatabaseToolkit
  - Automatically translate user query to SQL execution plan



AGENT

Thought

Ok, that seems to be a question to our database.

When it comes to databases I can choose from a range of possible actions (see below)

I should start collecting the name of all available tables.

sql_db_list_tables
(List tables in DB)

sql_db_schema
(Show table schema)

sql_db_query
(Run SQL query)

Database

Obervation

Observation: AGENTS, CUSTOMER, ORDERS

Thought

I should query the schema of the CUSTOMER table to see what columns I can use.

Action

Action: sql_db_schema
Action Input: CUSTOMER

AGENT

sql_db_list_tables
(List tables in DB)

sql_db_schema
(Show table schema)

sql_db_query
(Run SQL query)

Obervation

```
CREATE TABLE "CUSTOMER" (
 "CUST_CODE" TEXT(6) NOT NULL,
 "CUST_NAME" TEXT(40) NOT NULL,
 ....
)
```

Thought

I should query the CUSTOMER table to get the number of customers.

Action

Action: sql_db_query
Action Input: SELECT COUNT(*) FROM CUSTOMER

Database

Obervation

Observation: [(25,)]

Final Answer

We have 25 unique customers in our database.

Issues with ReAct

- Requires LLM call for each tool invocation
- Plans only 1 sub-problem at a time (step-by-step execution)
- Serial (blocking) execution
- Can not reason about whole task

- Motivates…

Plan and Execute agents

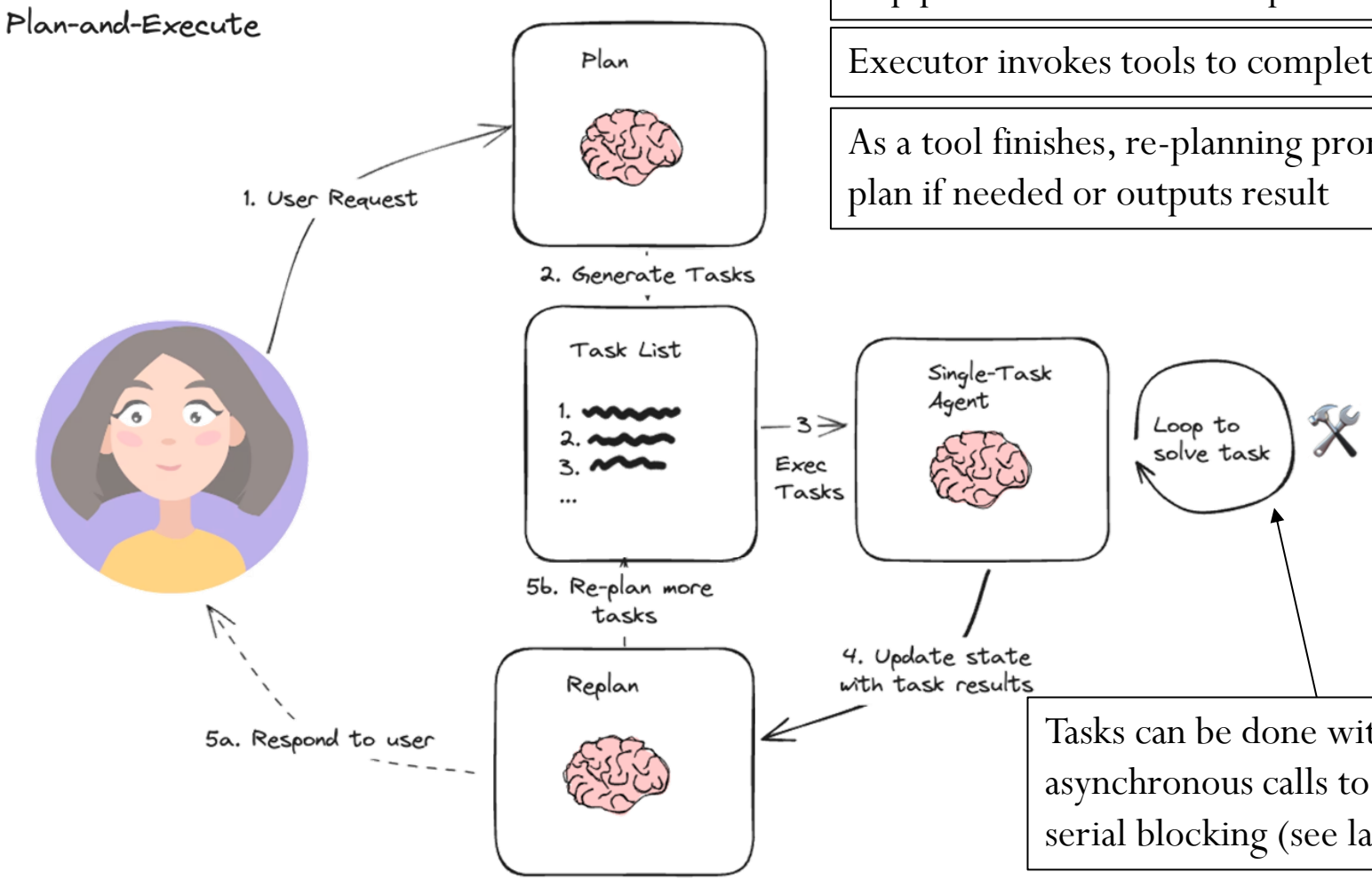
Plan-And-Execute

- Also, Plan-and-Solve Prompting and BabyAGI
  - LangChain walkthrough

Planner prompts an LLM to generate multi-step plan to answer user request

Executor invokes tools to complete tasks

As a tool finishes, re-planning prompt updates plan if needed or outputs result



Tasks can be done with asynchronous calls to tools to avoid serial blocking (see labs)

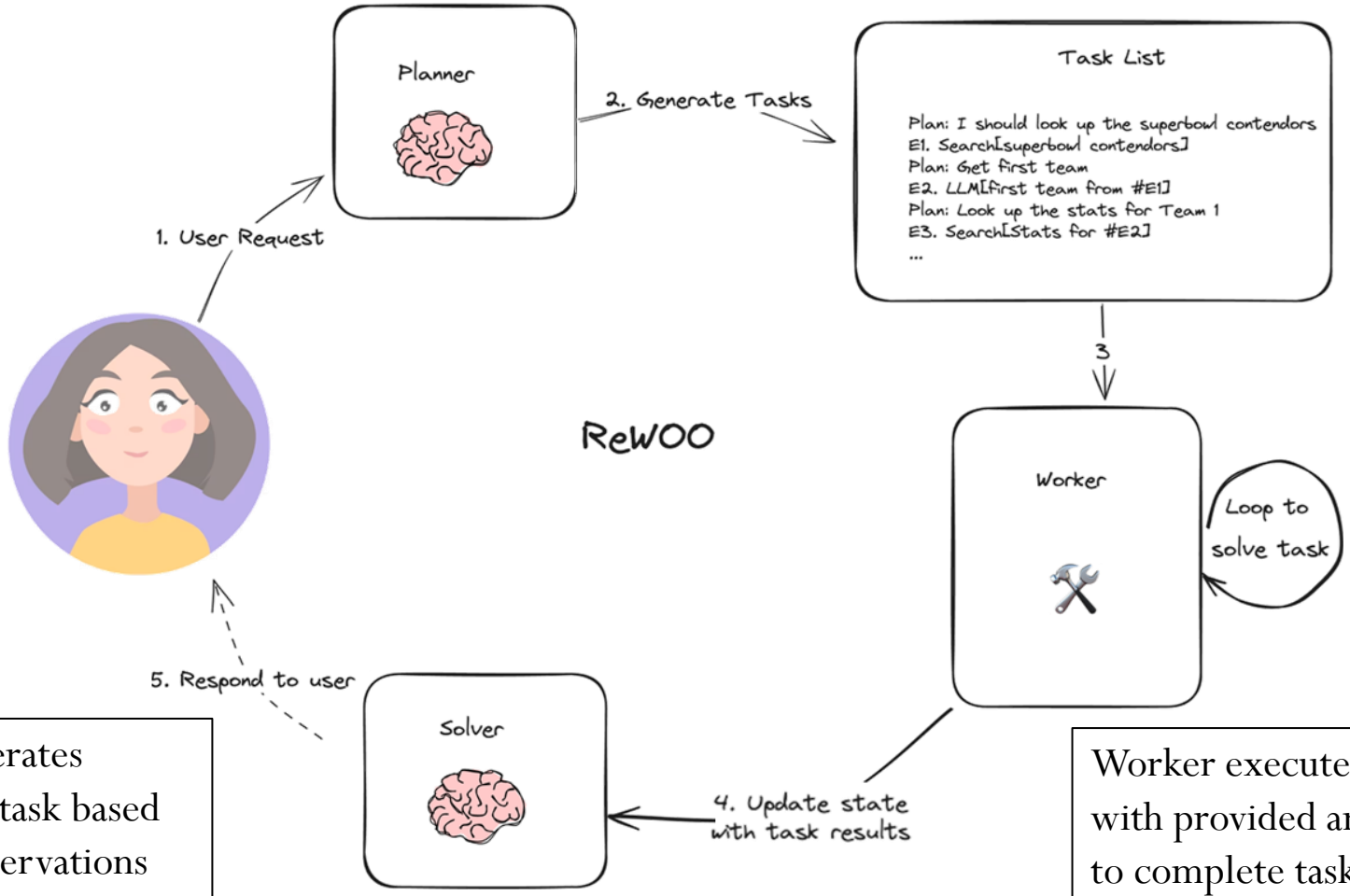
Plan-and-Execute prompt template

```
For the given objective, come up with a simple step by step plan.
This plan should involve individual tasks, that if executed correctly
will yield the correct answer. Do not add any superfluous steps.
The result of the final step should be the final answer. Make sure
that each step has all the information needed - do not skip steps.

Your objective was this:
{input}

Your original plan was this:
{plan}

You have currently done the following steps:
{past_steps}

Update your plan accordingly. If no more steps are needed and you can
return to the user, then respond with that. Otherwise, fill out the
plan. Only add steps to the plan that still NEED to be done. Do not
return previously done steps as part of the plan.
```

Reasoning WithOut Observations (ReWOO)

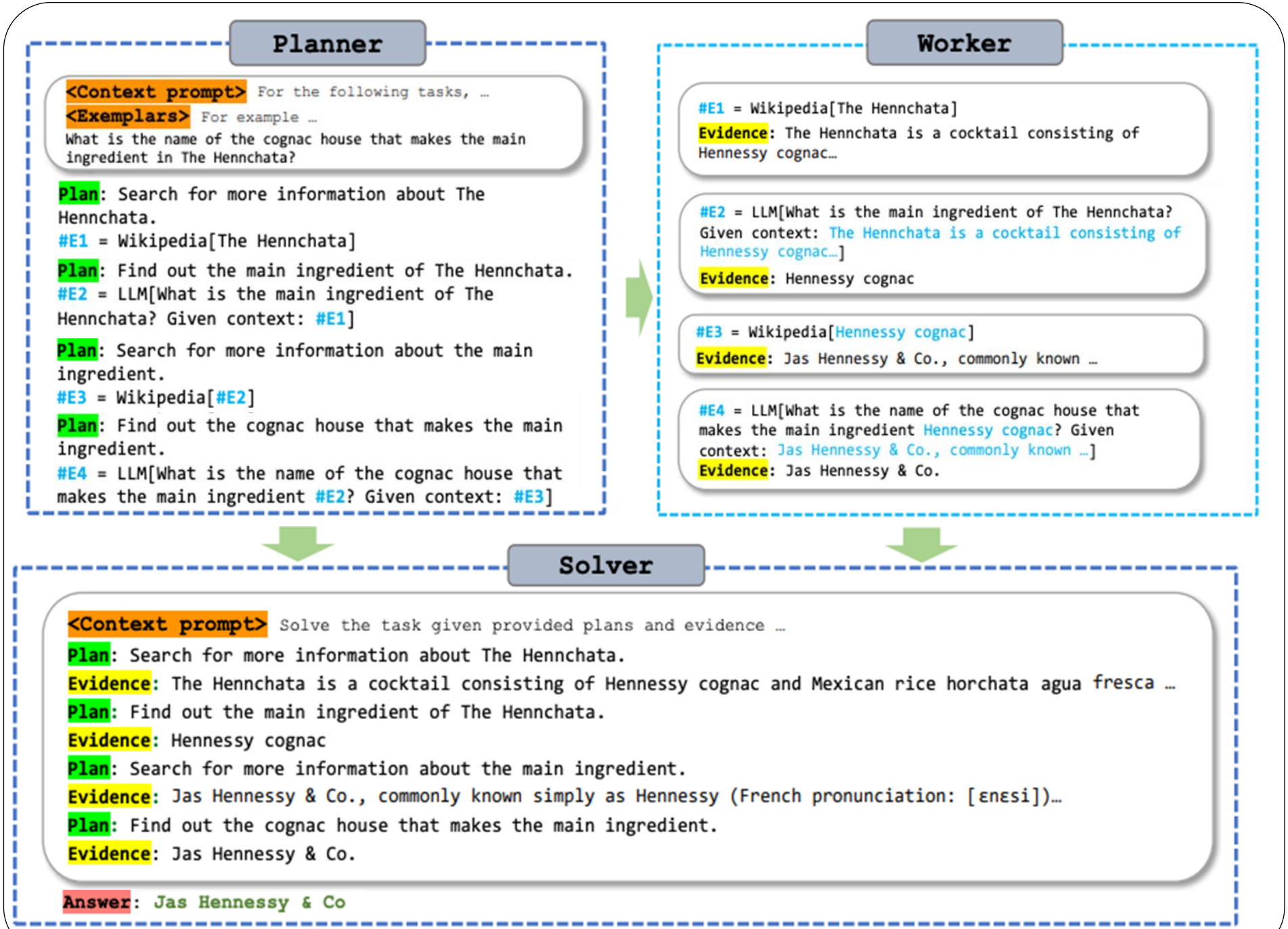
- Avoid per-task LLM calls

Planner prompts LLM for a structured multi-step plan to answer user request



Solver generates answer for task based on tool observations

Worker executes tools with provided arguments to complete tasks


Planner
<Context prompt> For the following tasks, …
<Exemplars> For example …
What is the name of the cognac house that makes the main ingredient in The Hennchata?
Plan: Search for more information about The Hennchata.
#E1 = Wikipedia[The Hennchata]
Plan: Find out the main ingredient of The Hennchata.
#E2 = LLM[What is the main ingredient of The Hennchata? Given context: #E1]
Plan: Search for more information about the main ingredient.
#E3 = Wikipedia[#E2]
Plan: Find out the cognac house that makes the main ingredient.
#E4 = LLM[What is the name of the cognac house that makes the main ingredient #E2? Given context: #E3]
Worker
#E1 = Wikipedia[The Hennchata]
Evidence: The Hennchata is a cocktail consisting of Hennessy cognac…
#E2 = LLM[What is the main ingredient of The Hennchata? Given context: The Hennchata is a cocktail consisting of Hennessy cognac…]
Evidence: Hennessy cognac
#E3 = Wikipedia[Hennessy cognac]
Evidence: Jas Hennessy & Co., commonly known …
#E4 = LLM[What is the name of the cognac house that makes the main ingredient Hennessy cognac? Given context: Jas Hennessy & Co., commonly known …]
Evidence: Jas Hennessy & Co.
Solver
<Context prompt> Solve the task given provided plans and evidence …
Plan: Search for more information about The Hennchata.
Evidence: The Hennchata is a cocktail consisting of Hennessy cognac and Mexican rice horchata agua fresca …
Plan: Find out the main ingredient of The Hennchata.
Evidence: Hennessy cognac
Plan: Search for more information about the main ingredient.
Evidence: Jas Hennessy & Co., commonly known simply as Hennessy (French pronunciation: [ɛnɛsi])…
Plan: Find out the cognac house that makes the main ingredient.
Evidence: Jas Hennessy & Co.
Answer: Jas Hennessy & Co

ReWoo prompt template

- Planner prompt

```
For the following task, make plans that can solve the problem step
by step. For each plan, indicate which external tool together with
tool input to retrieve evidence. You can store the evidence into a
variable #E that can be called by later tools. (Plan, #E1, Plan,
#E2, Plan, ...)

Tools can be one of the following:
(1) Google[input]: Worker that searches results from Google. Useful
when you need to find answers about a specific topic. The input
should be a search query.
(2) LLM[input]: A pretrained LLM like yourself. Useful when you need
to act with general world knowledge and common sense. Prioritize it
when you are confident in solving the problem yourself. Input can be
any instruction.

Begin!
Describe your plans with rich details. Each Plan should be followed
by only one #E.

Task: {task}"""
```

What is the hometown of the 2024 Australian Open winner?

```
Plan: Use Google to search for the 2024 Australian Open winner.
#E1 = Google[2024 Australian Open winner]

Plan: Retrieve the name of the 2024 Australian Open winner from the
search results.
#E2 = LLM[What is the name of the 2024 Australian Open winner,
given #E1]

Plan: Use Google to search for the hometown of the 2024 Australian
Open winner.
#E3 = Google[hometown of 2024 Australian Open winner, given #E2]

Plan: Retrieve the hometown of the 2024 Australian Open winner from
the search results.
#E4 = LLM[What is the hometown of the 2024 Australian Open winner,
given #E3]
```

- Solver prompt (after executions complete)

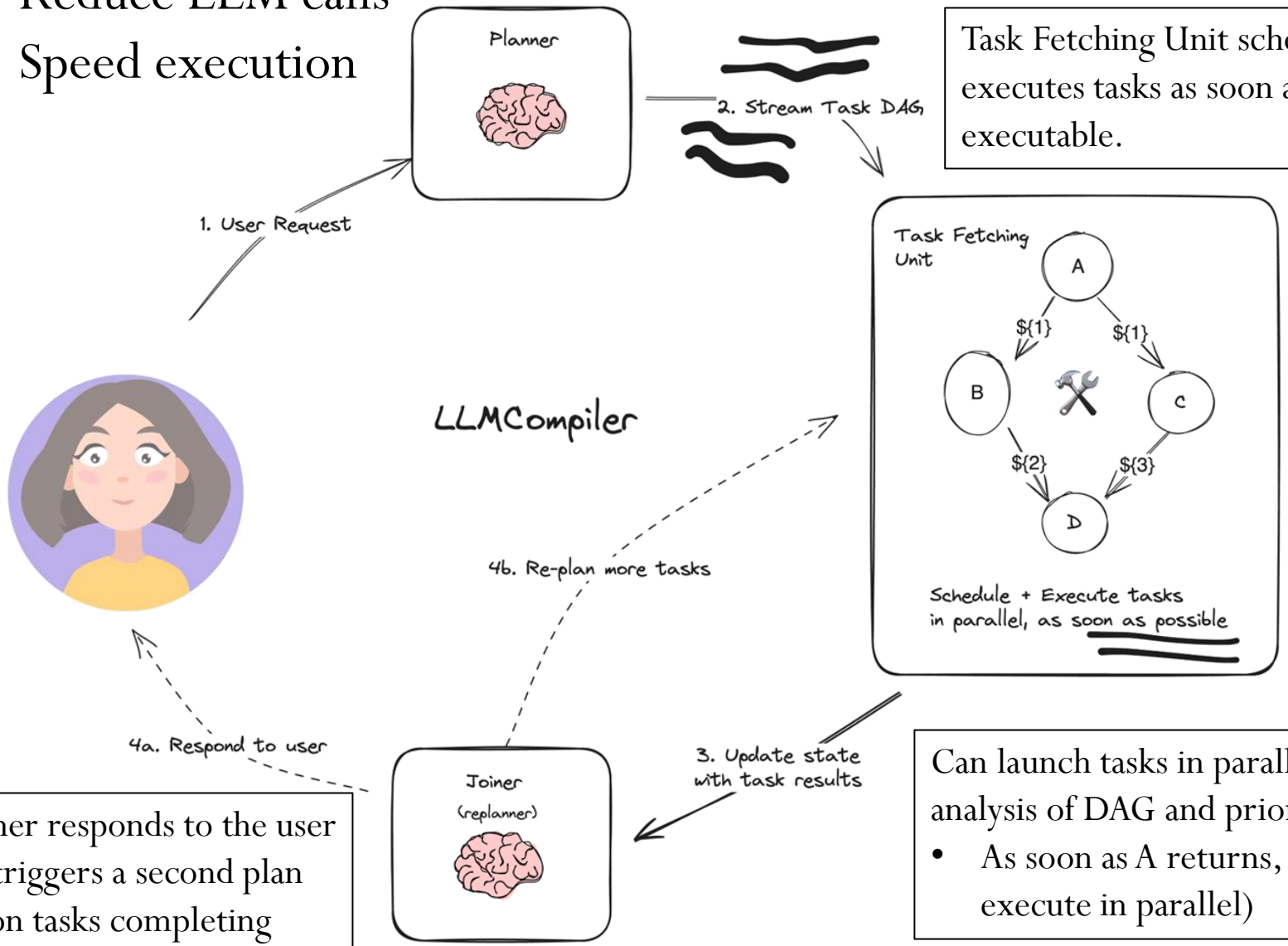
```
Solve the following task or problem. To solve the problem,
we have made a step-by-step Plan and retrieved
corresponding Evidence to each step. Use them with caution
since long evidence might contain irrelevant information.

{plan}

Now solve the question or task according to provided
Evidence above. Respond with the answer
directly with no extra words.

Task: {task}
Response:"""
```

LLMCompiler

- Reduce LLM calls
- Speed execution

Planner streams out DAG (directed acyclic graph) of tasks

Task Fetching Unit schedules and executes tasks as soon as they are executable.

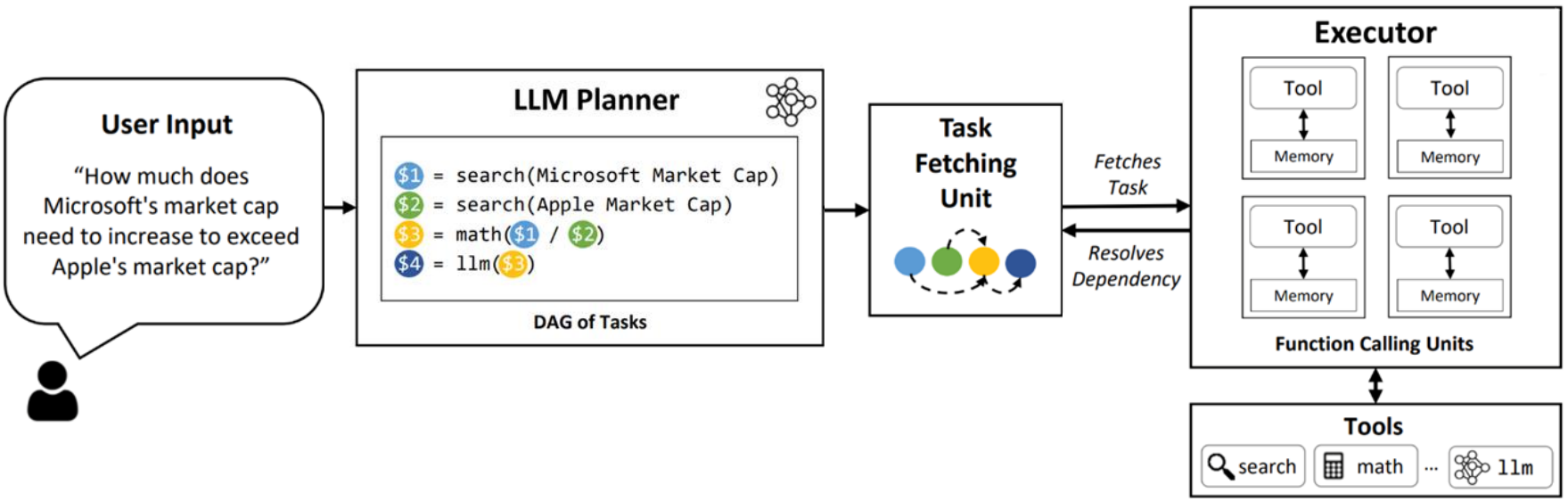


Joiner responds to the user or triggers a second plan upon tasks completing

Can launch tasks in parallel based on analysis of DAG and prior executions

- As soon as A returns, B and C can execute in parallel)

- Market cap query
  - Ask LLM to generate plan for answering query
  - Generate DAG of tasks and stream to TFU
  - TFU invokes Searches S1 and S2 in parallel, then Math tool called
  - Joiner sends result to LLM for final answer



LLMCompiler prompt template

- Planner prompt

```
Given a user query, create a plan to solve it with the utmost
parallelizability. Each plan should comprise an action from the
following {num_tools} types:
{tool_descriptions}
{num_tools}.
join(): Collects and combines results from prior actions.

 - An LLM agent is called upon invoking join() to either finalize
the user query or wait until the plans are executed.
 - join should always be the last action in the plan, and will be
called in two scenarios:
   (a) if the answer can be determined by gathering the outputs from
tasks to generate the final response.
   (b) if the answer cannot be determined in the planning phase
before you execute the plans.
```

Guidelines:
 - Each action described above contains input/output types and description.
    - You must strictly adhere to the input and output types for each action.
    - The action descriptions contain the guidelines. You MUST strictly follow those guidelines when you use the actions.
 - Each action in the plan should strictly be one of the above types. Follow the Python conventions for each action.
 - Each action MUST have a unique ID, which is strictly increasing.
 - Inputs for actions can either be constants or outputs from preceding actions. In the latter case, use the format $id to denote the ID of the previous action whose output will be the input.
 - Always call join as the last action in the plan. Say '<END_OF_PLAN>' after you call join
 - Ensure the plan maximizes parallelizability.
 - Only use the provided action types. If a query cannot be addressed using these, invoke the join action for the next steps.
 - Never introduce new actions other than the ones provided.

- Replanner

```
- You are given "Previous Plan" which is the plan that the
previous agent created along with the execution results
(given as Observation) of each plan and a general thought
(given as Thought) about the executed results
    You MUST use this information to create the next plan
under "Current Plan
  - When starting the Current Plan, you should start with
"Thought" that outlines the strategy for the next plan
  - In the Current Plan, you should NEVER repeat the actions
that are already executed in the Previous Plan
  - You must continue the task index from the end of the
previous one. Do not repeat task indices.
```

Aside: Models and function calling

- Models must accurately format function calls to invoke tools
  - Benchmarks for determining best models to do so
  - Berkeley Function Calling Leaderboard

BFCL: From Tool Use to Agentic Evaluation of Large Language Models

The Berkeley Function Calling Leaderboard (BFCL) V4 evaluates the LLM's ability to call functions (aka tools) accurately. This leaderboard consists of real-world data and will be updated periodically. For more information on the evaluation dataset and methodology, please refer to our blogs: BFCL-v1 introducing AST as an evaluation metric, BFCL-v2 introducing enterprise and OSS-contributed functions, BFCL-v3 introducing multi-turn interactions, and BFCL-v4 introducing holistic agentic evaluation. Checkout code and data.

Last Updated: 2025-12-16 *[Change Log]*

Search model names... Search

				Agentic		Multi Turn	Single Turn		Hallucination Measureme
				Web Search ▶	Memory ▶	Multi turn ▶	Non-live (AST) ▶	Live (AST) ▶	
Rank	Overall Acc	Model	Cost ($)	Overall Acc	Overall Acc	Overall Acc	Overall Acc	Overall Acc	Relevance	Irrelevanc
1	77.47	Claude-Opus-4-5-20251101 (FC)	86.55	84.5	73.76	68.38	88.58	79.79	62.5	84.72
2	73.24	Claude-Sonnet-4-5-20250929 (FC)	43.73	81	64.95	61.37	88.65	81.13	68.75	86.61
3	72.51	Gemini-3-Pro-Preview (Prompt)	298.47	80	61.72	60.75	90.65	83.12	68.75	85.59
4	72.38	GLM-4.6 (FC thinking)	4.64	77.5	55.7	68	87.56	80.9	75	84.96
5	69.57	Grok-4-1-fast-reasoning (FC)	17.26	82.5	53.98	58.87	88.27	78.46	81.25	79.43

Code agents

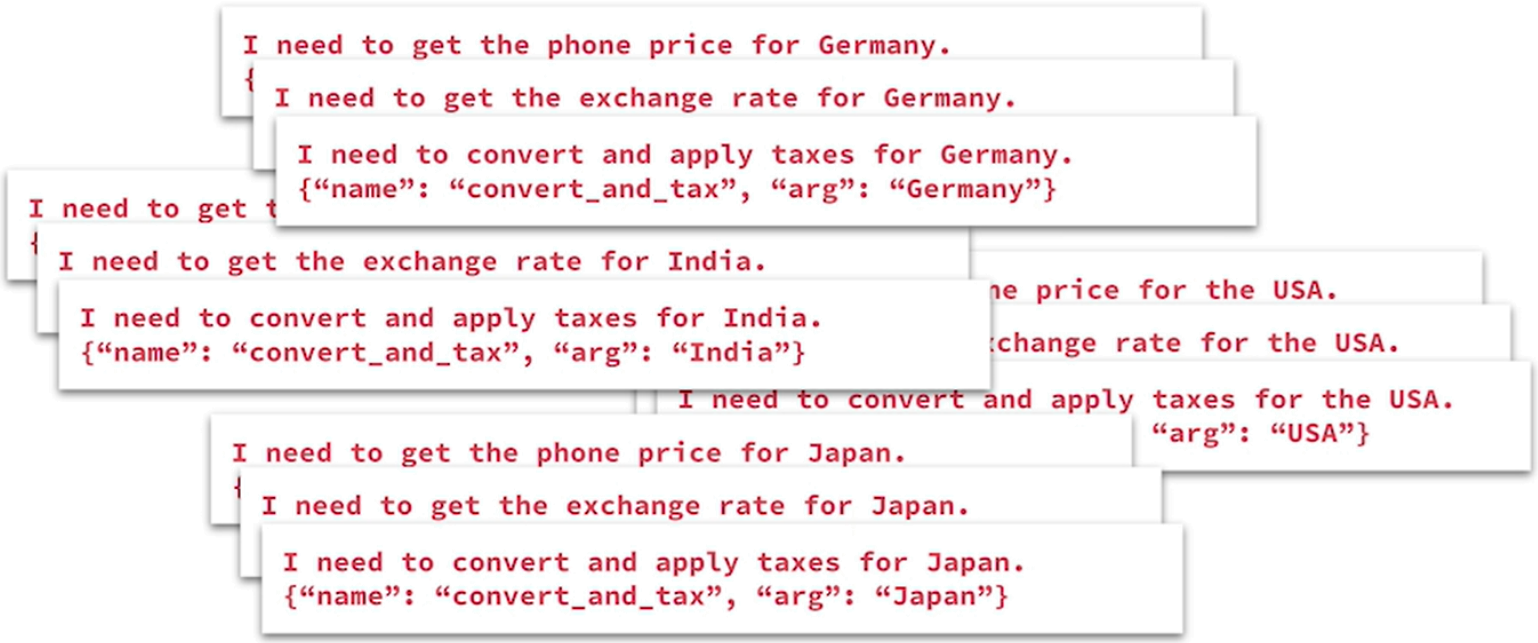
Code agents

- Allow agent to produce code that is executed
  - "Executable Code Actions Elicit Better LLM Agents"

```
Task:  Determine the most cost-effective country to purchase
       a smartphone: USA, Japan, Germany, or India
```

  - Tool-calling approach (12 agent tool calls)

```
Tools: get_phone_price, get_rate,  convert_and_tax
```



- Code agent
  - e.g. Has one tool (PythonREPL)
  - One tool call produces all code to calculate answer

````
I need to get final prices for each country and compare them.
```py
prices = {}
for country in ["USA", "Japan", "Germany", "India"]:
    price = get_phone_price(country)
    exchange_rate = get_rate(country)
    prices[country] = convert_and_tax(price, country, exchange_rate)

best_country = min(prices, key=prices.get)
final_answer(f"Best country: {best_country}, Price: ${prices[best_country]}")
```
````

Reasoning models

Recall: Common reasoning tasks

- Reasoning tasks (Week 1)
  - Deductive
  - Inductive
  - Mathematical
  - Temporal
  - Spatial
  - Common-sense
  - Causal
  - Multi-hop
  - Analogical

- Agents requiring multi-hop planning and reasoning over many tasks
  - But, initial models not specifically tuned for planning and reasoning

Planning benchmarks

Results on PlanBench as of 6/20/2024

Domain		Claude-3.5 -Sonnet	Claude 3 (Opus)	GPT-4o	GPT-4	GPT-4-Turbo	Gemini Pro	LLaMA-3 70B
Blocksworld	One shot	346/600 (57.6%)	289/600 (48.1%)	170/600 (28.3%)	206/600 (34.3%)	138/600 (23%)	68/600 (11.3%)	76/600 (12.6%)
	Zero shot	329/600 (54.8%)	356/600 (59.3%)	213/600 (35.5%)	210/600 (34.6%)	241/600 (40.1%)	3/600 (0.5%)	205/600 (34.16%)
Mystery Blocksworld	One shot	19/600 (3.1%)	8/600 (1.3%)	5/600 (0.83%)	26/600 (4.3%)	5/600 (0.83%)	2/500 (0.4%)	15/600 (2.5%)
	Zero shot	0/600 (0%)	0/600 (0%)	0/600 (0%)	1/600 (0.16%)	1/600 (0.16%)	0/500 (0%)	0/600 (0%)

- Can one build better agents with a model specifically trained to reason?

Leads to...

What Is Going On Inside OpenAIs Strawberry (o1)?

Vishal Rajput · Follow

Published in AIGuys · 12 min read · Sep 15, 2024

- Focus less on training and more on inferencing (e.g. thinking before answering)
  - Allow more time and computational budget on reasoning tasks
  - https://x.com/DrJimFan/status/1834279865933332752

- Factor out reasoning from knowledge
  - Model tuned for reasoning tasks

- Example: Tree-of-Thoughts
  - Devote resources to try many approaches, then select best

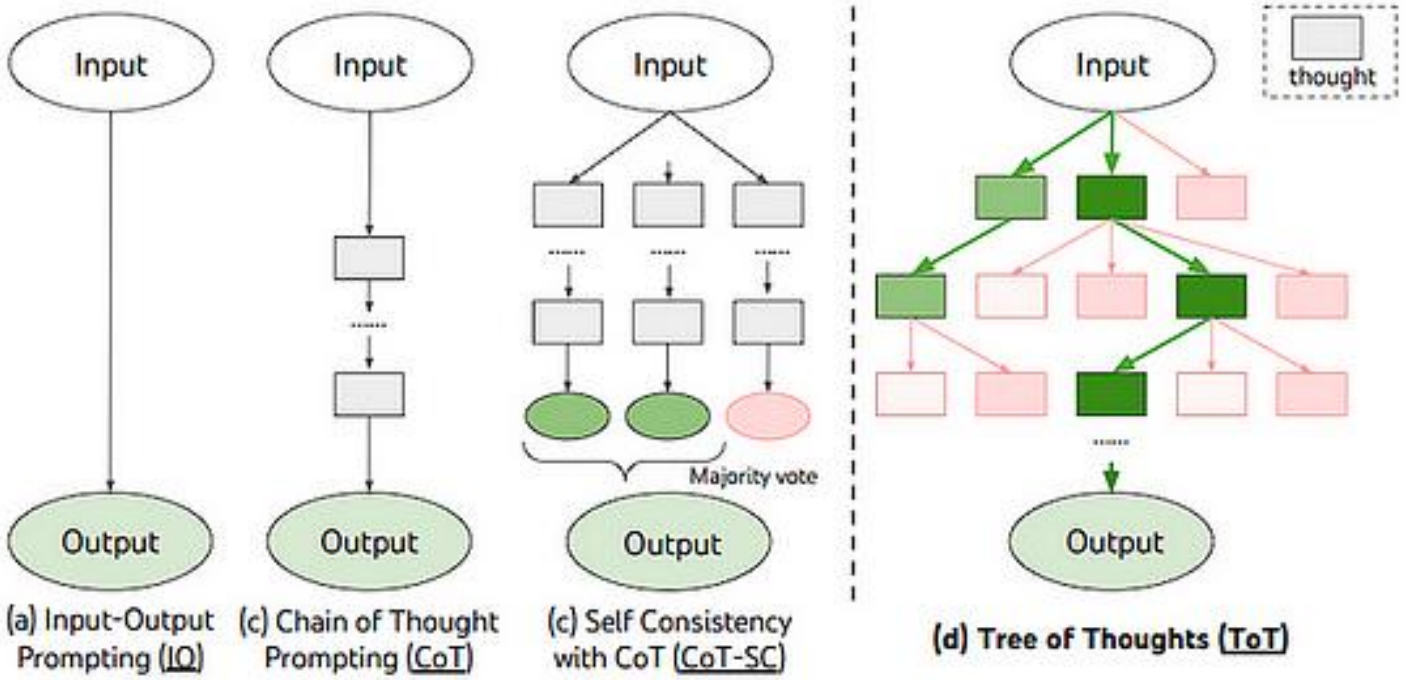
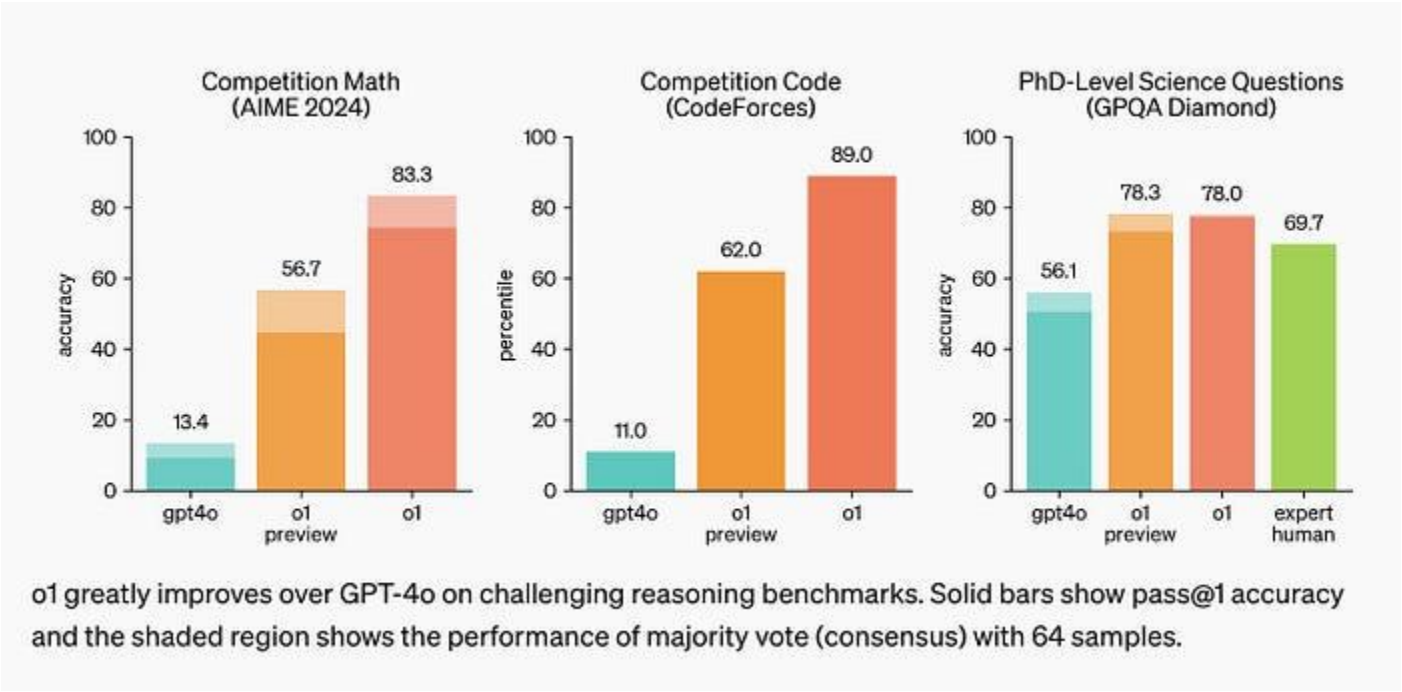


Figure 1: Schematic illustrating various approaches to problem solving with LLMs. Each rectangle box represents a *thought*, which is a coherent language sequence that serves as an intermediate step toward problem solving. See concrete examples of how thoughts are generated, evaluated, and searched in Figures 2,4,6.

- Result?



o1 greatly improves over GPT-4o on challenging reasoning benchmarks. Solid bars show pass@1 accuracy and the shaded region shows the performance of majority vote (consensus) with 64 samples.

Opinions mixed

Apple study exposes deep cracks in LLMs' "reasoning" capabilities

Irrelevant red herrings lead to "catastrophic" failure of logical inference.

KYLE ORLAND – OCT 14, 2024 2:21 PM | 632

"Current LLMs are not capable of genuine logical reasoning. Instead, they attempt to replicate the reasoning steps observed in their training data."

Google DeepMind CEO Demis Hassabis may have just dropped 'bombshell' for OpenAI CEO Sam Altman

TOI Tech Desk / TIMESOFINDIA.COM / Updated: Jan 20, 2026, 11:21 IST

Preferred on G | 4 Comments | Share | AA

"Today's large language models are phenomenal at pattern recognition but they don't truly understand causality. They don't really know why A leads to B. They just predict the next token based on statistical correlations."

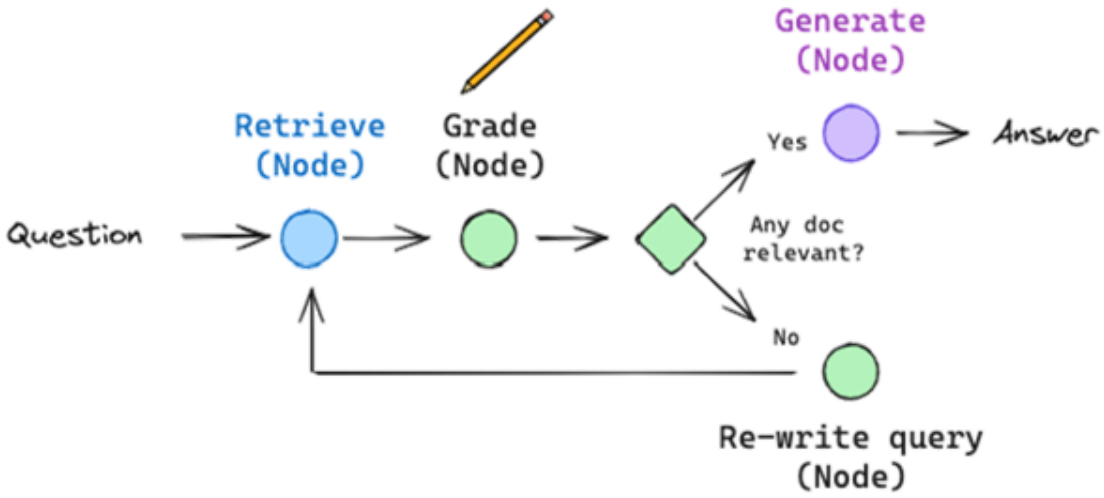
"Real scientific invention requires something deeper: the ability to run thought experiments, simulate physics accurately, and reason from first principles. That needs a world model—an internal simulation engine that understands how reality actually works, not just how words typically follow each other."

- We'll find out

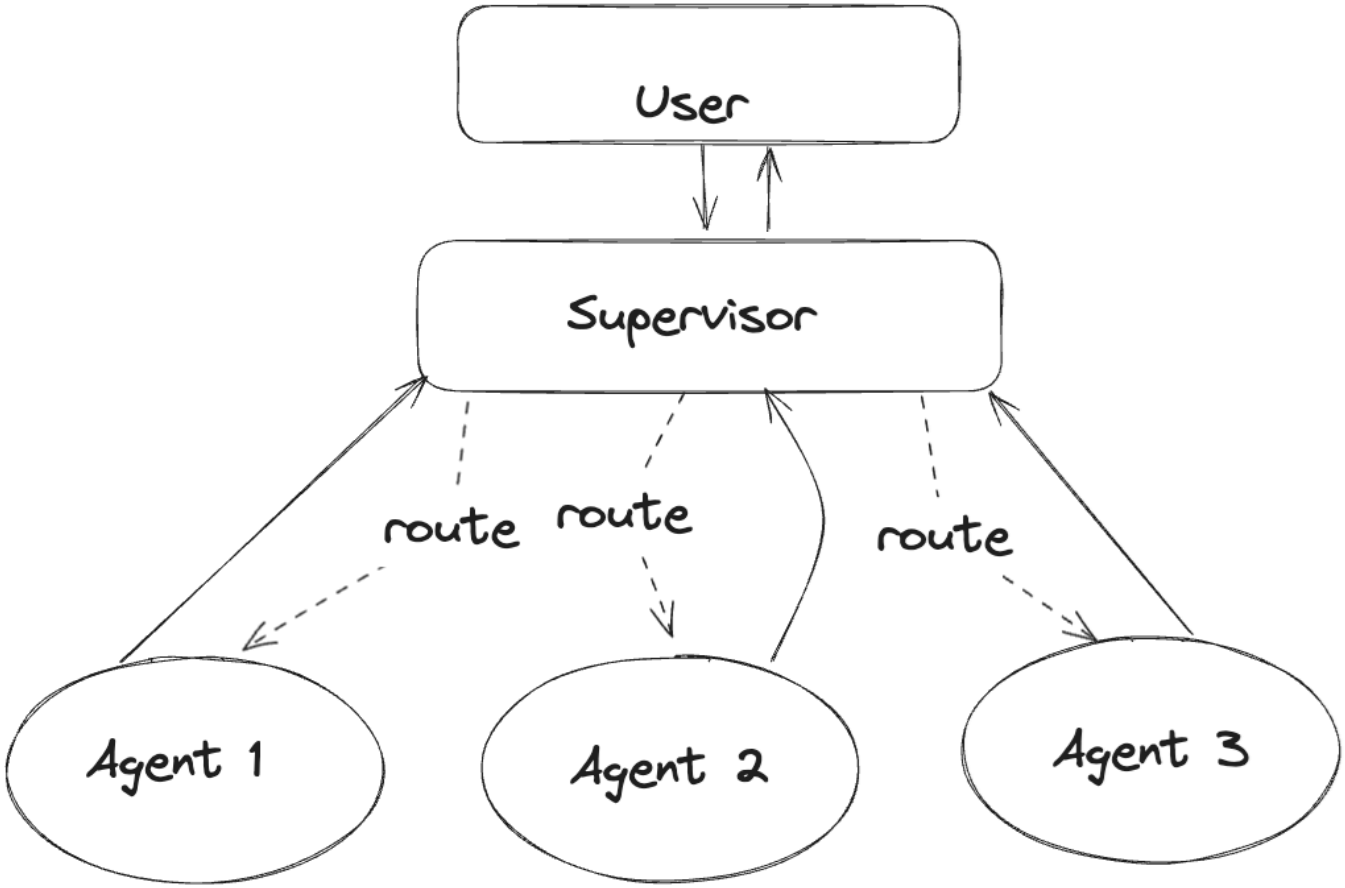
LangGraph

LangGraph

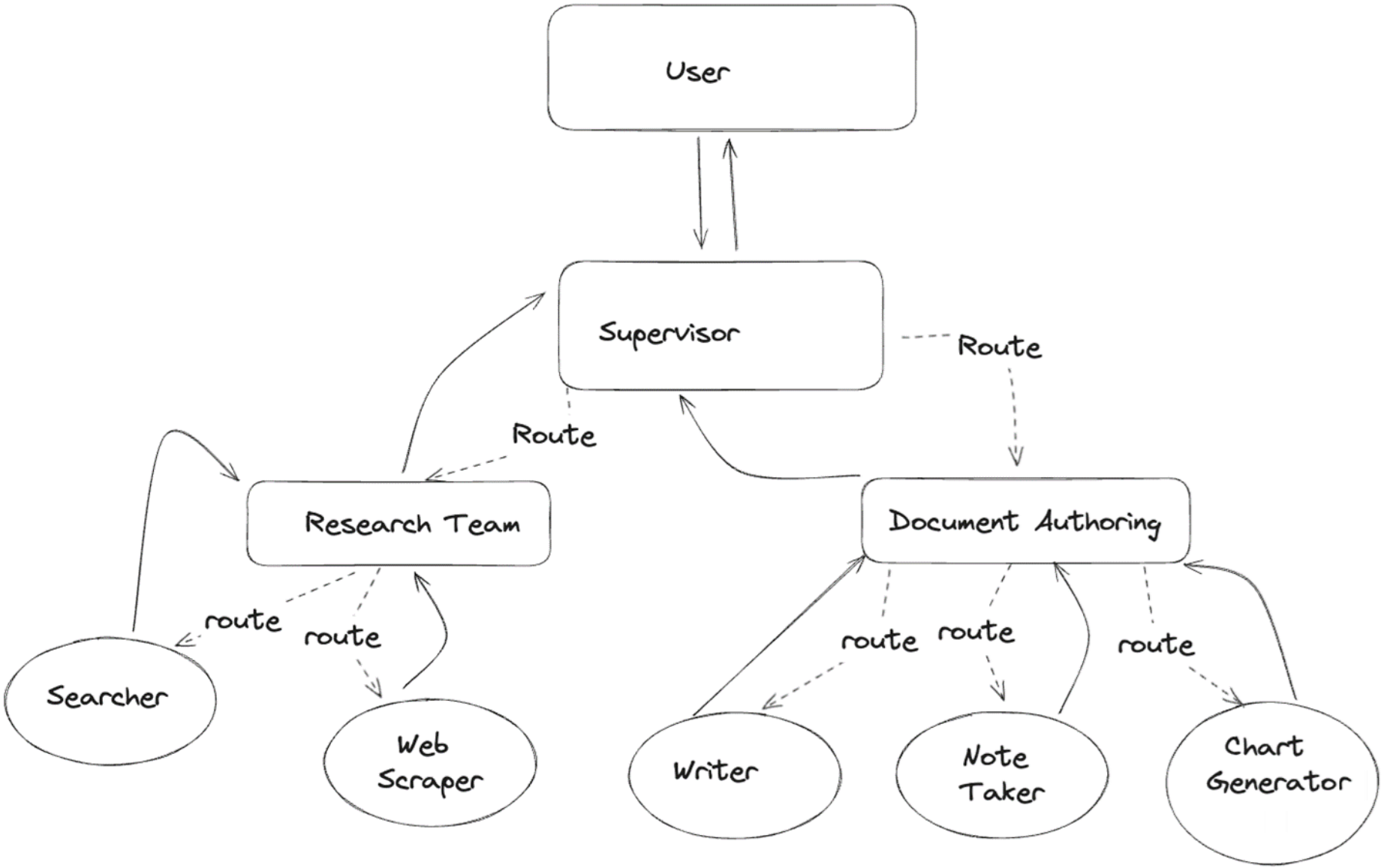
- Framework for building stateful, multi-agent applications
  - Prior examples with potentially parallel execution paths
    - Enable parallelism by allowing multiple, simultaneous execution of agents
  - Advanced applications have cycles of feedback in workflows
    - Must be able to change plans at run-time based on messages and state
- Both can be supported by expressing workload as a graph
- Example
  - Self-reflective RAG allowing self correction



- Agent supervisor
  - Supervisor agent utilizes other agents as tools



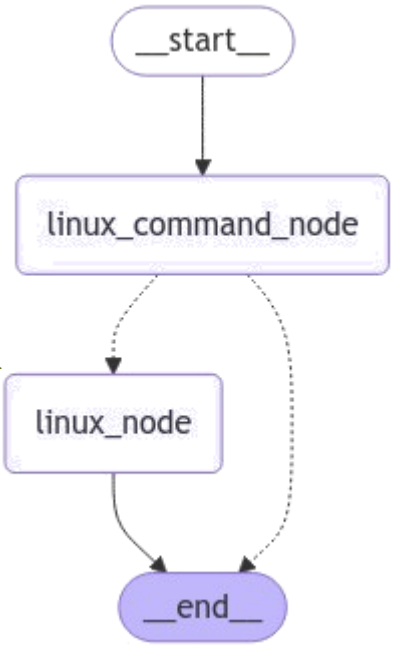
- Hierarchy of agents



- Example: Simple human-in-the-loop Linux command interface
  - 2 nodes: generate command, execute command
  - Human feedback check

```
def linux_command_node(user_input):
    response = llm.invoke(
        f"""Given the user's prompt, you are to generate a Linux command
to answer it. Provide no other formatting, just the command. \n\n  User
prompt: {user_input}""")
    return(response)

def user_check(linux_command):
    print(f"Linux command is: {linux_command}")
    user_ack = input("Should I execute this command? ")
    response = llm.invoke(f"""The following is the response the user gave
to 'Should I execute this command?': {user_ack} \n\n  If the answer given
is a negative one, return NO else return YES""")
    if 'YES' in response:
        return('linux_node')
    else:
        return(END)

def linux_node(linux_command):
    result = os.system(linux_command)
    return(END)
```

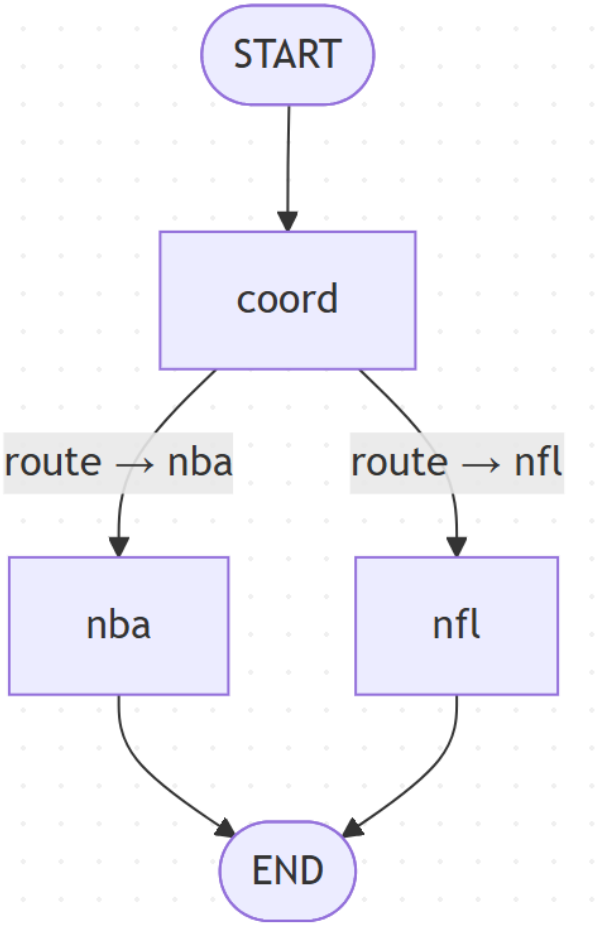
```
workflow = Graph()

workflow.add_node("linux_command_node", linux_command_node)
workflow.add_node("linux_node", linux_node)

workflow.add_edge(START, 'linux_command_node')
workflow.add_conditional_edges(
     'linux_command_node', user_check
)
workflow.add_edge('linux_node', END)
app = workflow.compile()

from IPython.display import Image, display
display(Image(app.get_graph().draw_mermaid_png()))
```



```
>> what is the size of the current directory in megabytes?
Linux command is: du -sh . | awk '{print $1}'

Should I execute this command? yes
294M
__end__
>> find all files that begin with the letter a
Linux command is: find . -name "a*"

Should I execute this command? no
find . -name "a*"

>>
```

LangGraph web assistant

- Spawn multiple agents to collect today's new headlines
  - Coordinator agent calls route to determine which assistant agents to call (NBA news agent, NFL news agent, or both)



- Route determines which agent(s) to call

```
async def route(state: State):
    prompt = f"""User request: {state['query']}
                 Decide which sports news is requested.
                 Reply with exactly one word: nba nfl both
             """
    resp = await llm.ainvoke(prompt)
    decision = resp.content.lower().strip()

    if decision == "nba":
        return Send("nba", state)
    if decision == "nfl":
        return Send("nfl", state)
    return [Send("nba", state), Send("nfl", state)]
```

- Nodes spun up to perform task

```
async def nfl_agent(state: State):
    html = await fetch("https://www.espn.com/espn/rss/nfl/news")
    summary = await extract_and_summarize(html)
    return {"nfl": summary}
async def nba_agent(state: State):
    html = await fetch("https://www.espn.com/espn/rss/nba/news")
    summary = await extract_and_summarize(html)
    return {"nba": summary}
```

- Each node asynchronously retrieves news feed and invokes summary tool

```
async def fetch(url: str) -> str:
    async with httpx.AsyncClient(timeout=15) as client:
        r = await client.get(url)
        return r.text

async def extract_and_summarize(html: str) -> str:
    prompt = f"""
You are given raw HTML from a sports headline feed.
1. Identify the main news article headlines on the page.
2. Ignore navigation, ads, and footers.
3. Summarize the current news in ONE concise paragraph.
4. Return only that paragraph as a string

HTML: {html[:12000]}
"""
    resp = await llm.ainvoke(prompt)
    return resp.content.strip()
```

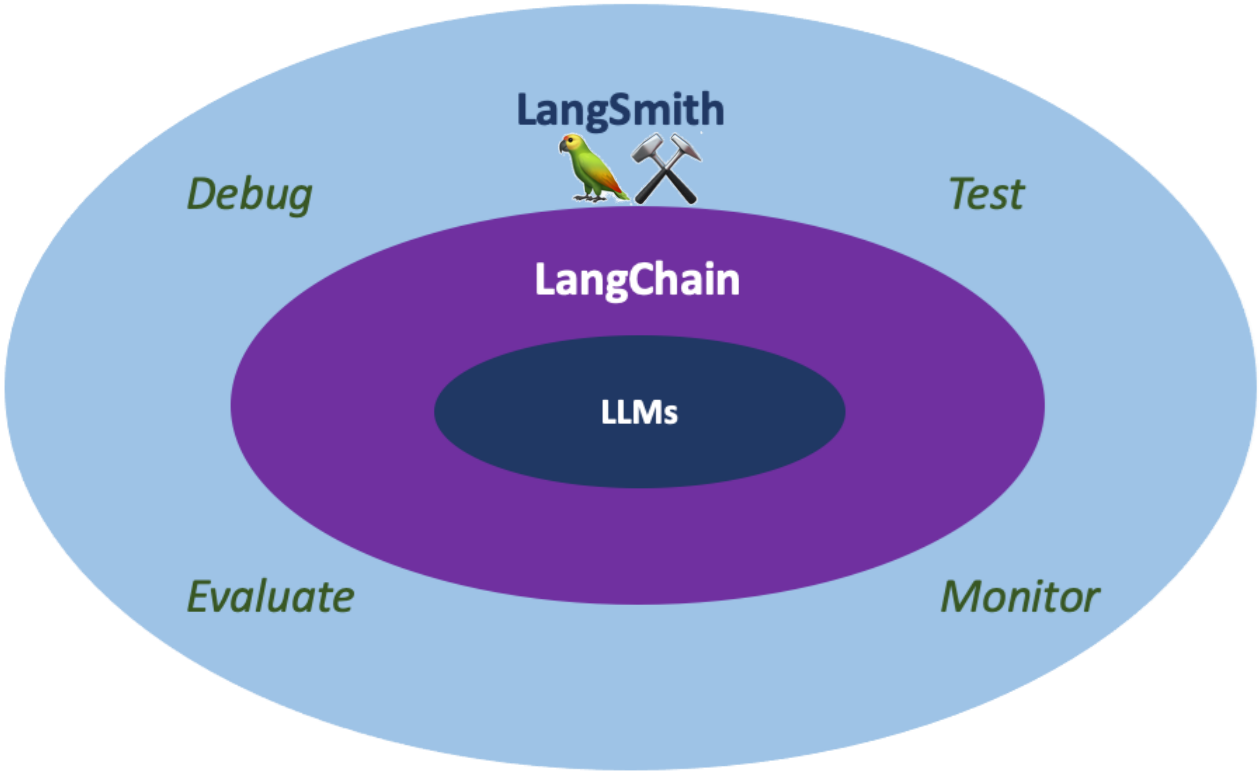
Other multi-agent frameworks

- CrewAI
- BeeAI, Agent Communication Protocol or ACP (IBM)
  - Standard agent-to-agent communication protocol for multi-agent workflows
- Agent2Agent protocol or A2A (Google)
  - Communication protocol to enable connection of multiple disparate agents
- Options for your homework if you wish to use

LangSmith

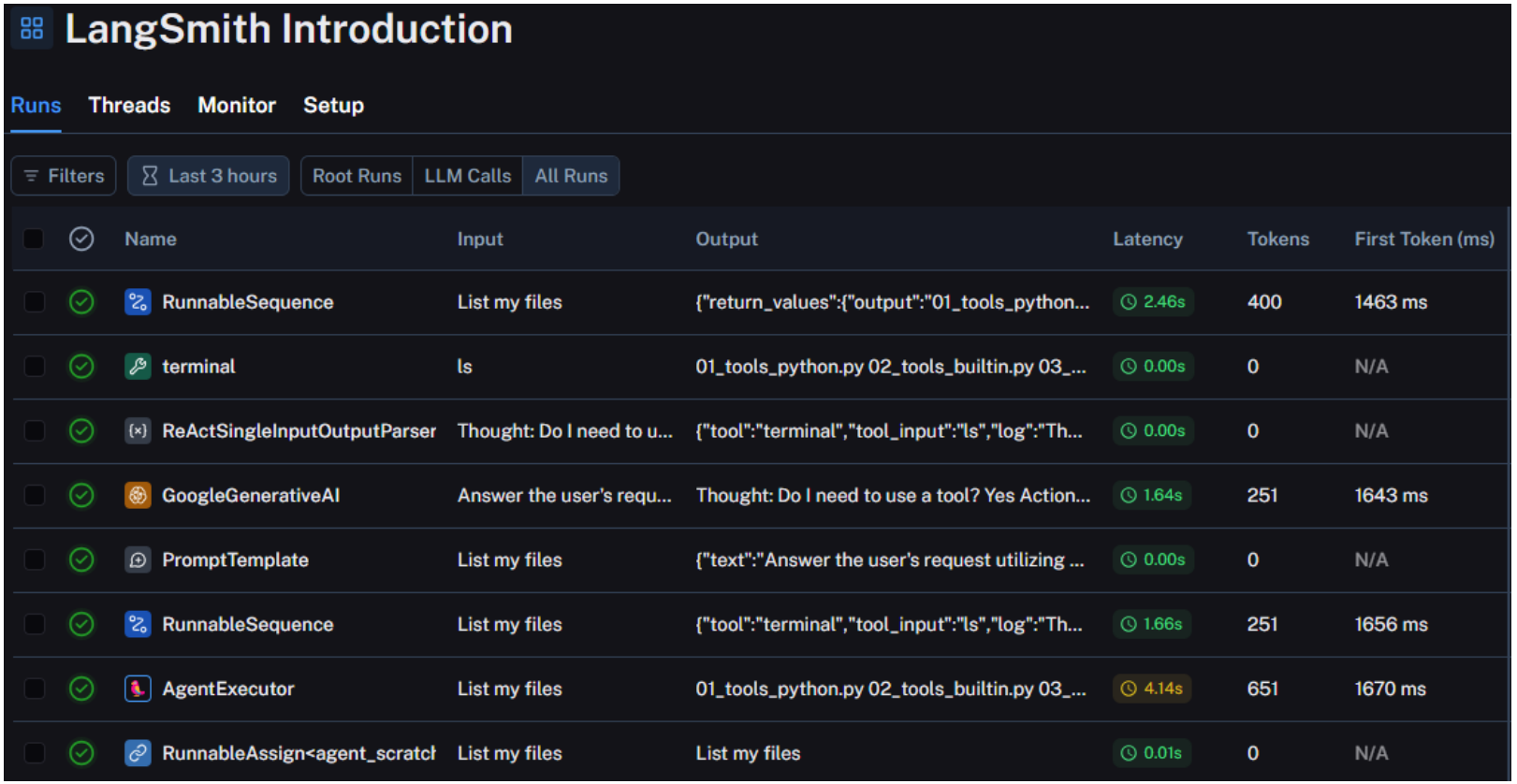
LangSmith

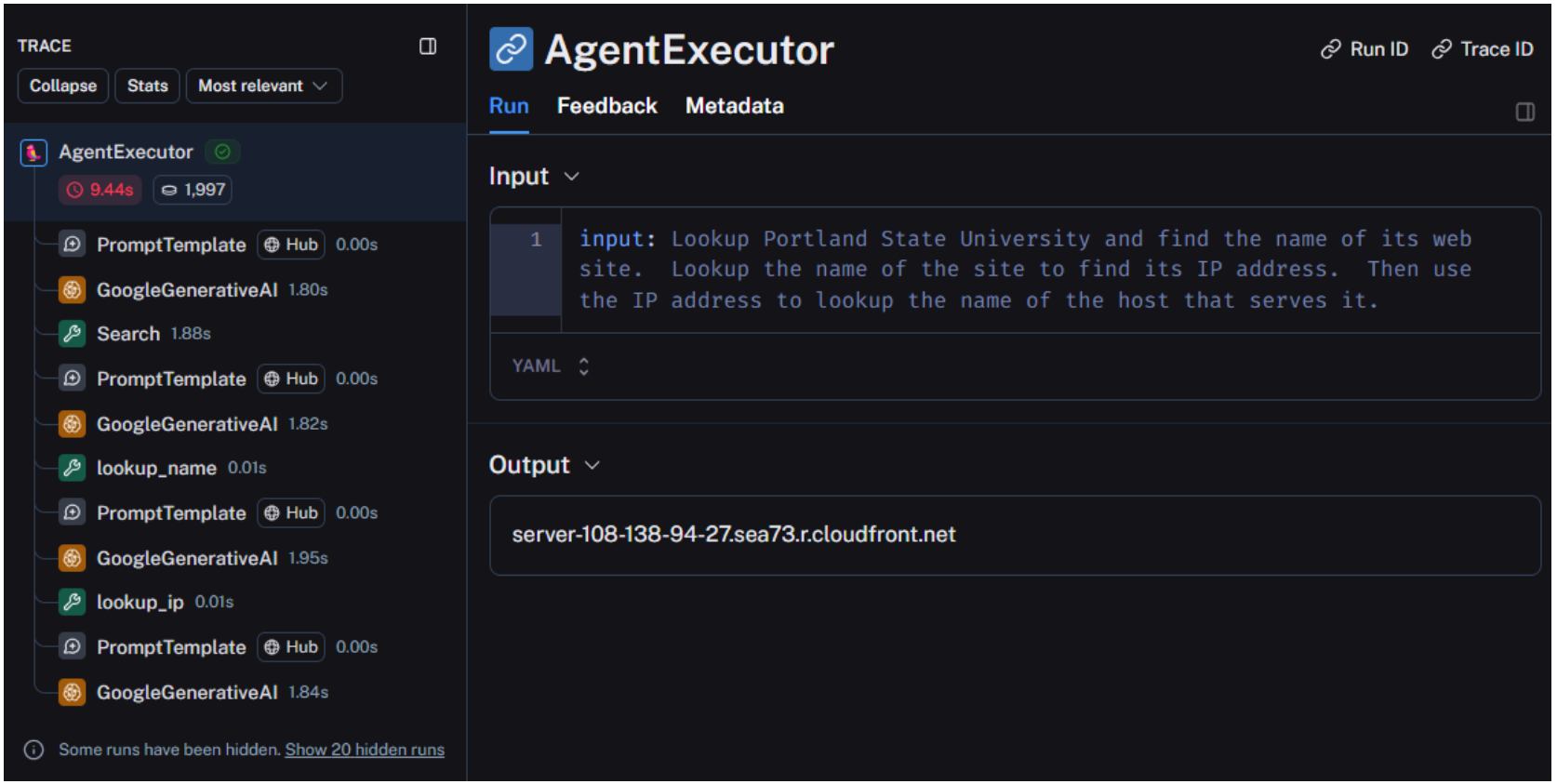
- Debugging and monitoring framework for LangChain applications
  - Traces of execution streamed to LangChain server



Tracing agent execution

- Examining
  - Chain/model/tool invocations
  - Execution time
  - Tokens

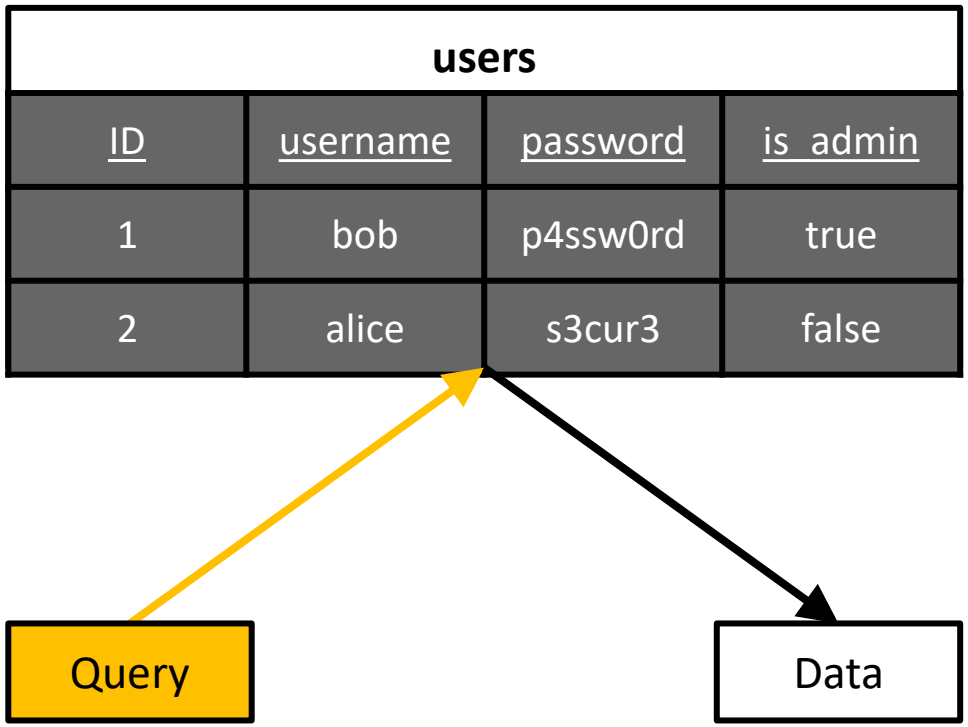



TRACE
Collapse
Stats
Most relevant
AgentExecutor
9.44s
1,997
PromptTemplate Hub 0.00s
GoogleGenerativeAI 1.80s
Search 1.88s
PromptTemplate Hub 0.00s
GoogleGenerativeAI 1.82s
lookup_name 0.01s
PromptTemplate Hub 0.00s
GoogleGenerativeAI 1.95s
lookup_ip 0.01s
PromptTemplate Hub 0.00s
GoogleGenerativeAI 1.84s
Some runs have been hidden. Show 20 hidden runs
AgentExecutor
Run ID
Trace ID
Run
Feedback
Metadata
Input
1
input: Lookup Portland State University and find the name of its web site. Lookup the name of the site to find its IP address. Then use the IP address to lookup the name of the host that serves it.
YAML
Output
server-108-138-94-27.sea73.r.cloudfront.net

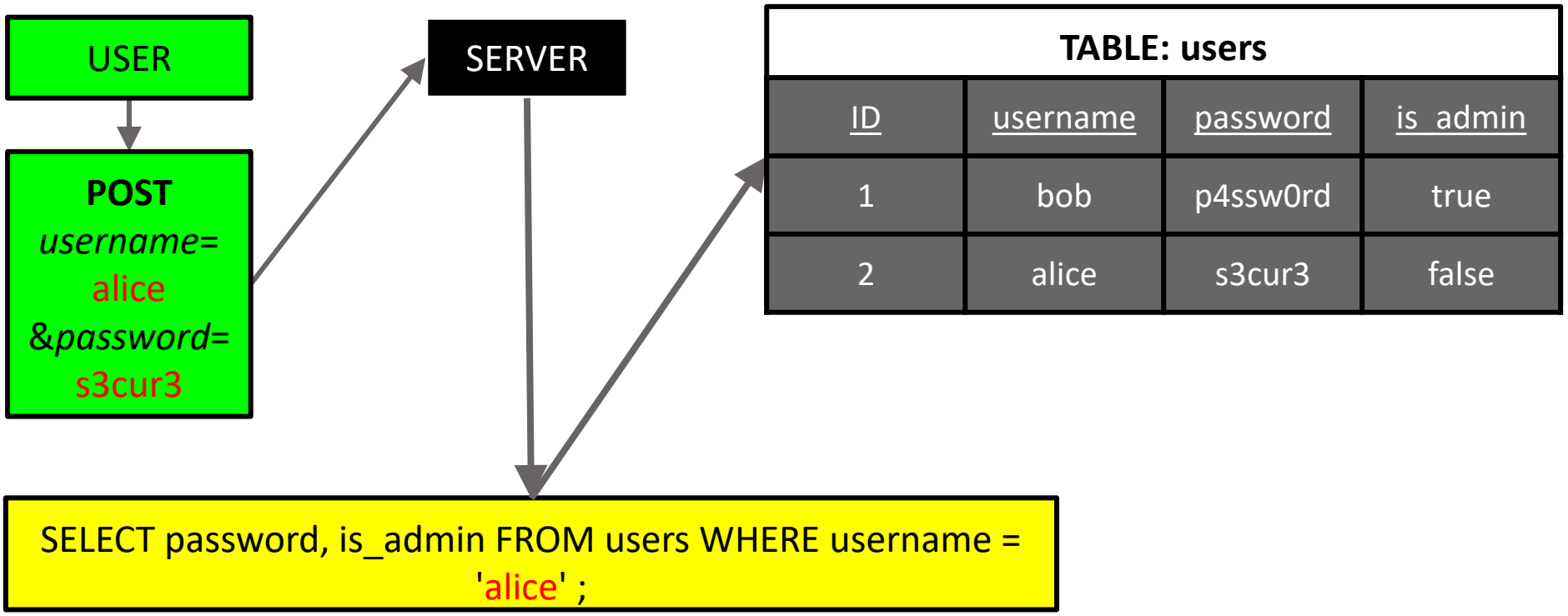
SQLi slides (for lab)

SQL commands

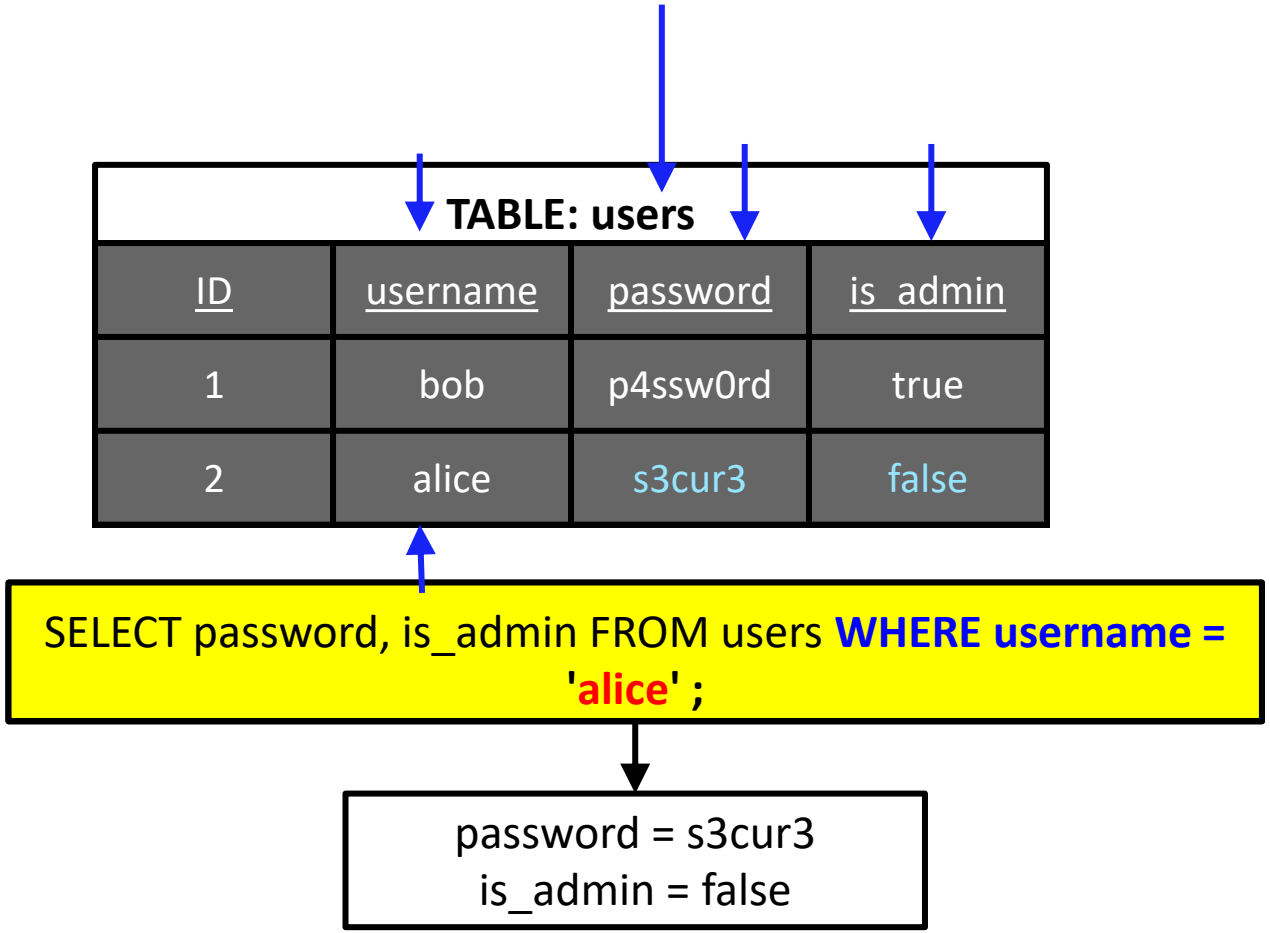
- Querying

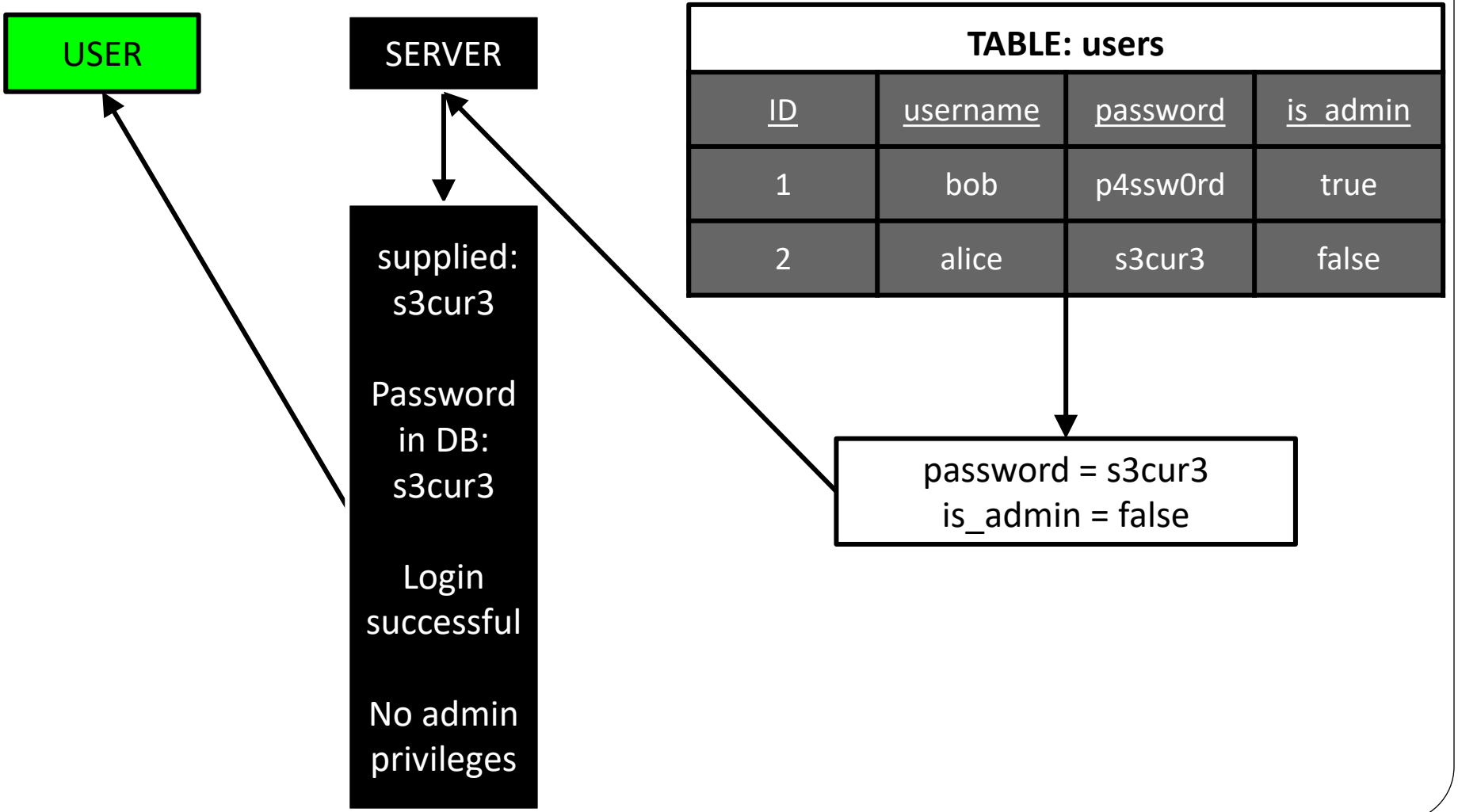


- Done via `SELECT`, `FROM`, and `WHERE`
  - Example: Logging in using SQL
  - Web application with query string template to fill in (note that parameter delimited via single quotes)



Dissecting the query string




USER
SERVER
TABLE: users
ID
username
password
is_admin
1
bob
p4ssw0rd
true
2
alice
s3cur3
false
supplied: s3cur3
Password in DB: s3cur3
Login successful
No admin privileges
password = s3cur3
is_admin = false

The perfect password (or username) ...

```
X' or '1'='1' --
```

✓ Uppercase letter

✓ Lowercase letter

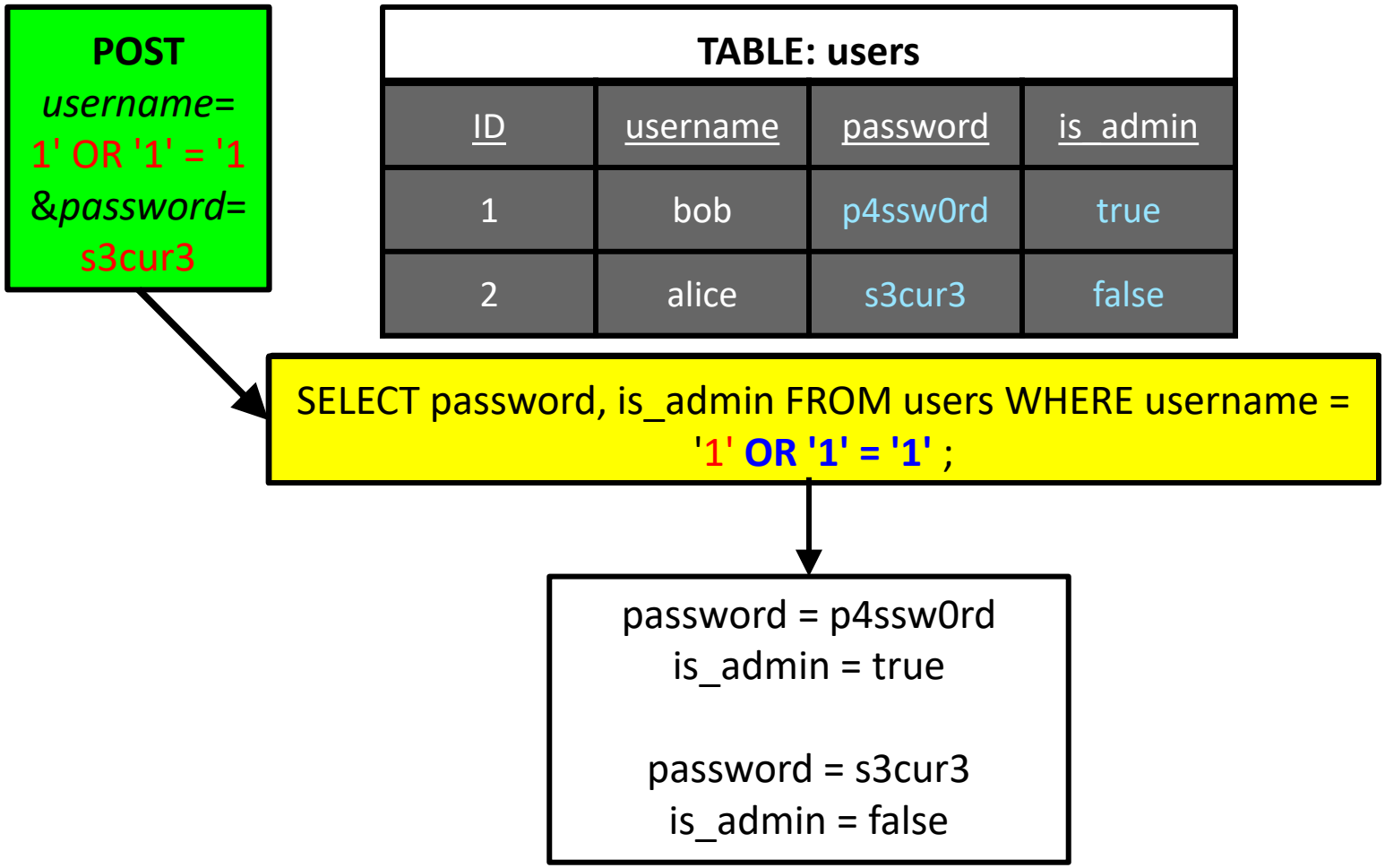
✓ Number

✓ Special character

✓ 16 characters

Also logs you into all sorts of things!

Basic SQL Injection



Homework #2

- Build-a-bot
  - Create your own custom LangChain agent
  - Minimum requirements
    - Utilize an alternate agent type or architecture than those used in exercises
    - Utilize one additional built-in tool or toolkit not contained in lab exercises
    - Create one custom tool or toolkit (you may also add your RAG chain as a custom tool)
    - Retain the Terminal or PythonREPL tool in your agent
- Same submission method

LLM agents

- What could go wrong?
  - Stay tuned